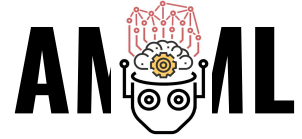




**Politecnico
di Torino**



Advanced Machine Learning

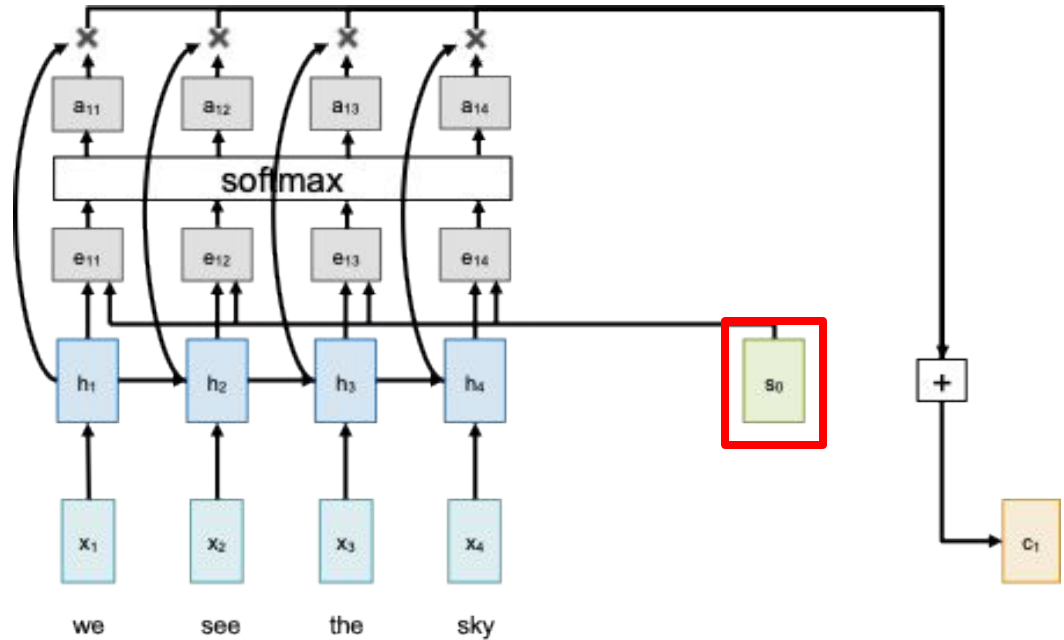
A.A. 2025/2026

Tatiana Tommasi

Attention Layer

Inputs:

Query vector: $\mathbf{q}$ [D_Q]

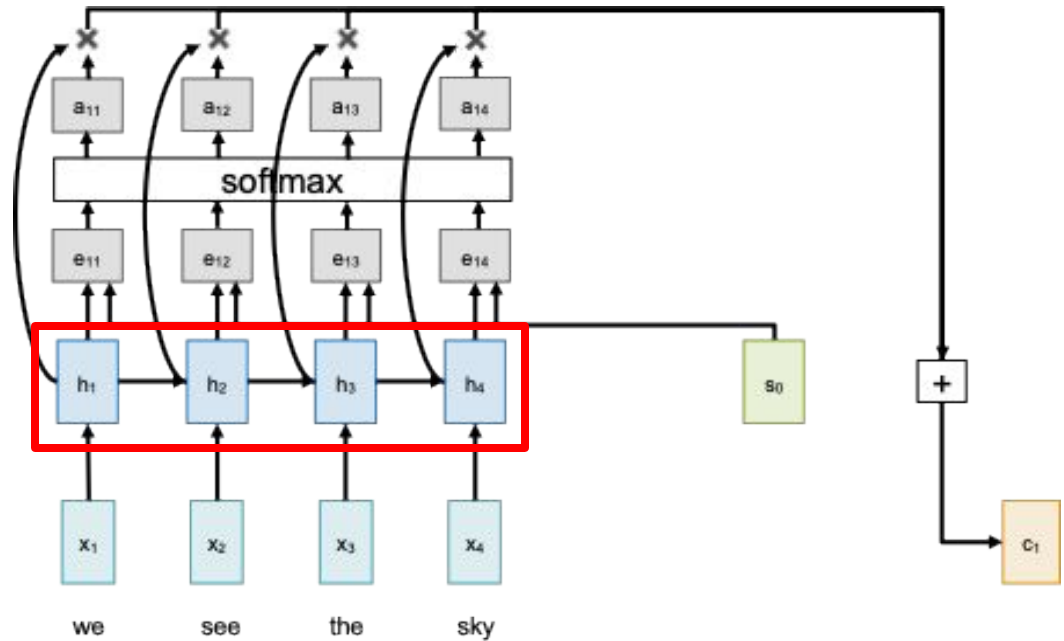


Attention Layer

Inputs:

Query vector: $\mathbf{q}$ [D_Q]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

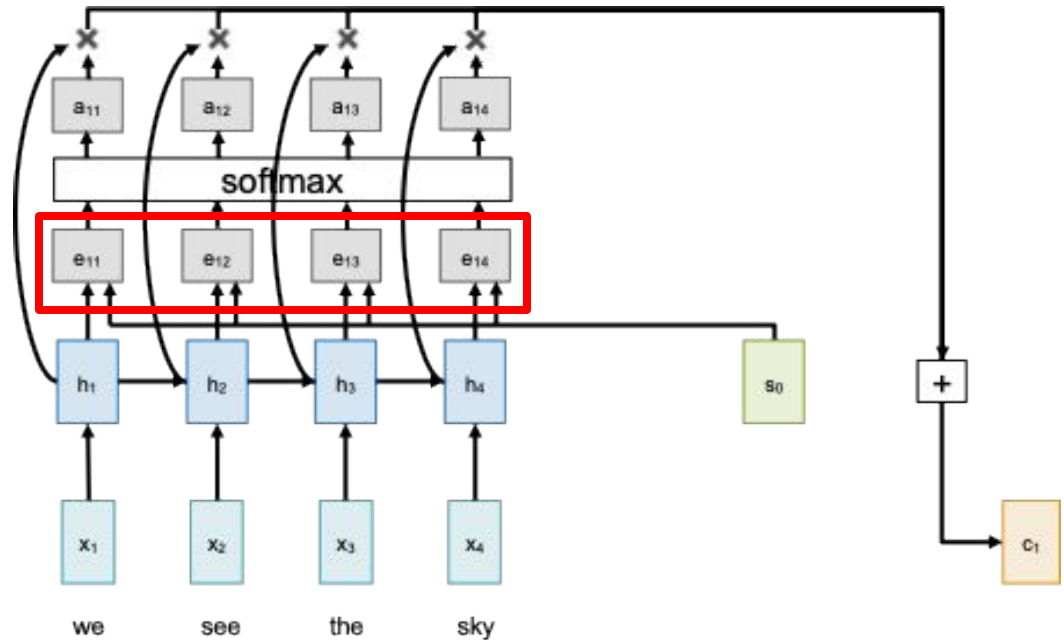


Attention Layer

Inputs:

Query vector: $\mathbf{q}$ [D_Q]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]



Computation:

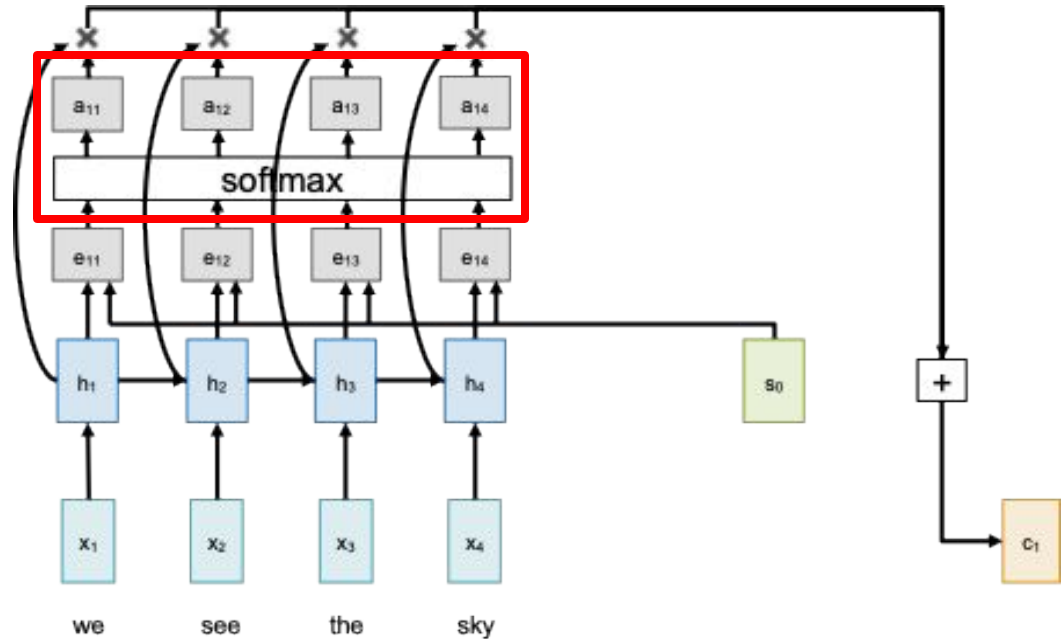
Similarities: e [N_X] $e_i = f_{\text{att}}(\mathbf{q}, \mathbf{X}_i)$

Attention Layer

Inputs:

Query vector: $\mathbf{q}$ [D_Q]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]



Computation:

Similarities: e [N_X] $e_i = f_{\text{att}}(\mathbf{q}, \mathbf{X}_i)$

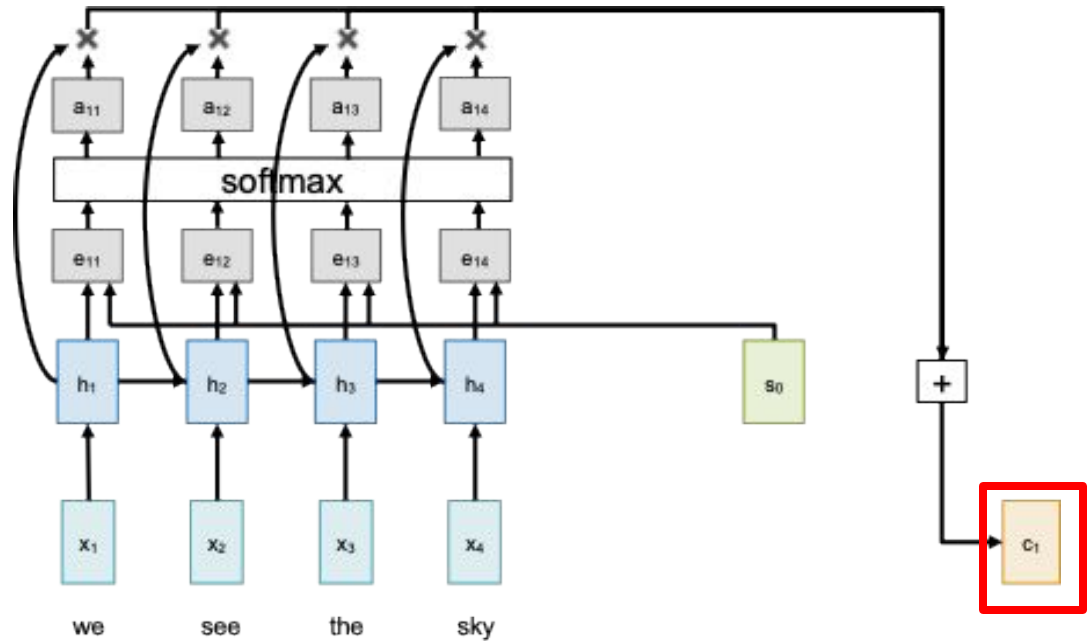
Attention weights: $a = \text{softmax}(e)$ [N_X]

Attention Layer

Inputs:

Query vector: $\mathbf{q}$ [D_Q]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]



Computation:

Similarities: e [N_X] $e_i = f_{\text{att}}(\mathbf{q}, \mathbf{X}_i)$

Attention weights: $\mathbf{a} = \text{softmax}(\mathbf{e})$ [N_X]

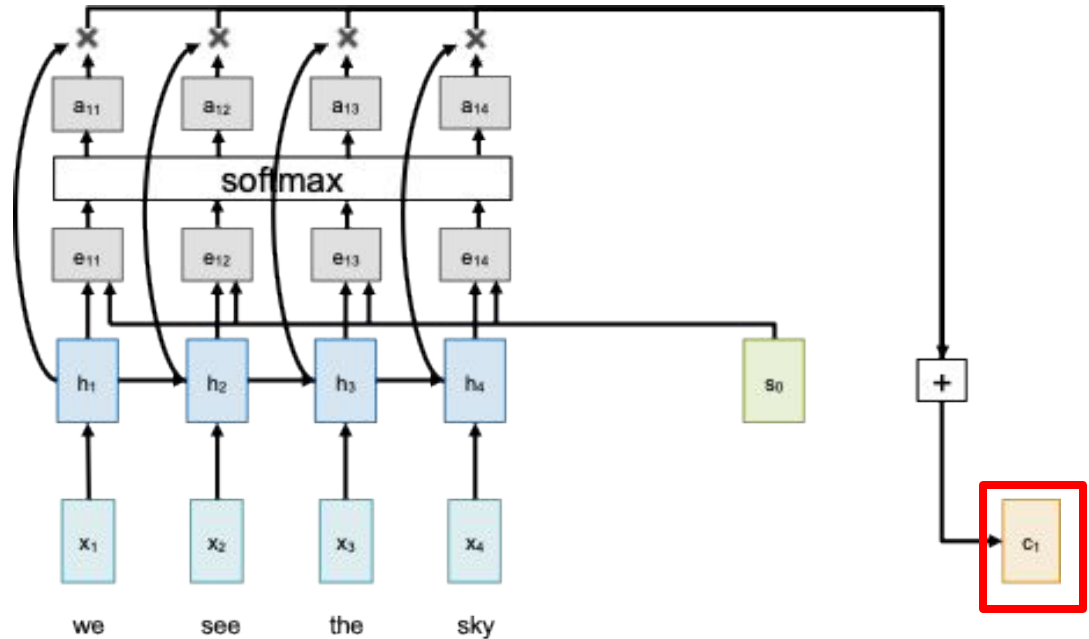
Output vector: $\mathbf{y} = \sum_i \mathbf{a}_i \mathbf{X}_i$ [D_X]

Attention Layer

Inputs:

Query vector: $\mathbf{q}$ [D_Q]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]



Computation:

Similarities: e [N_X] $e_i = f_{\text{att}}(\mathbf{q}, \mathbf{X}_i)$

Attention weights: $\mathbf{a} = \text{softmax}(\mathbf{e})$ [N_X]

Output vector: $\mathbf{y} = \sum_i \mathbf{a}_i \mathbf{X}_i$ [D_X]

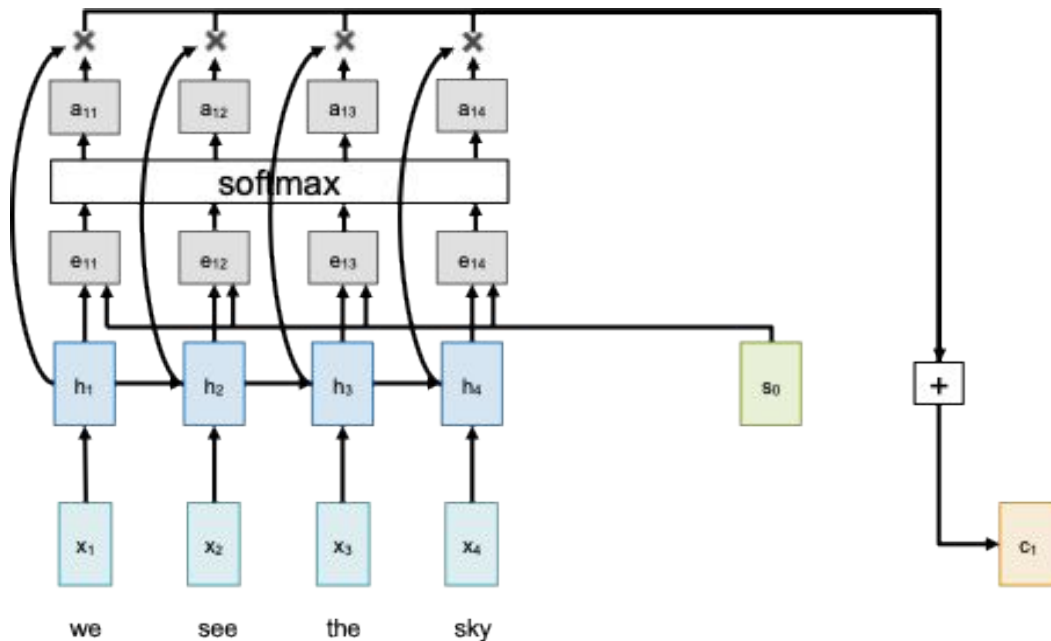
Let's generalize this!

Attention Layer

Inputs:

Query vector: $\mathbf{q}$ [D_X]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]



Computation:

Similarities: e [N_X] $e_i = \mathbf{q} \cdot \mathbf{X}_i$

Attention weights: $a = \text{softmax}(e)$ [N_X]

Output vector: $\mathbf{y} = \sum_i a_i \mathbf{X}_i$ [D_X]

Changes

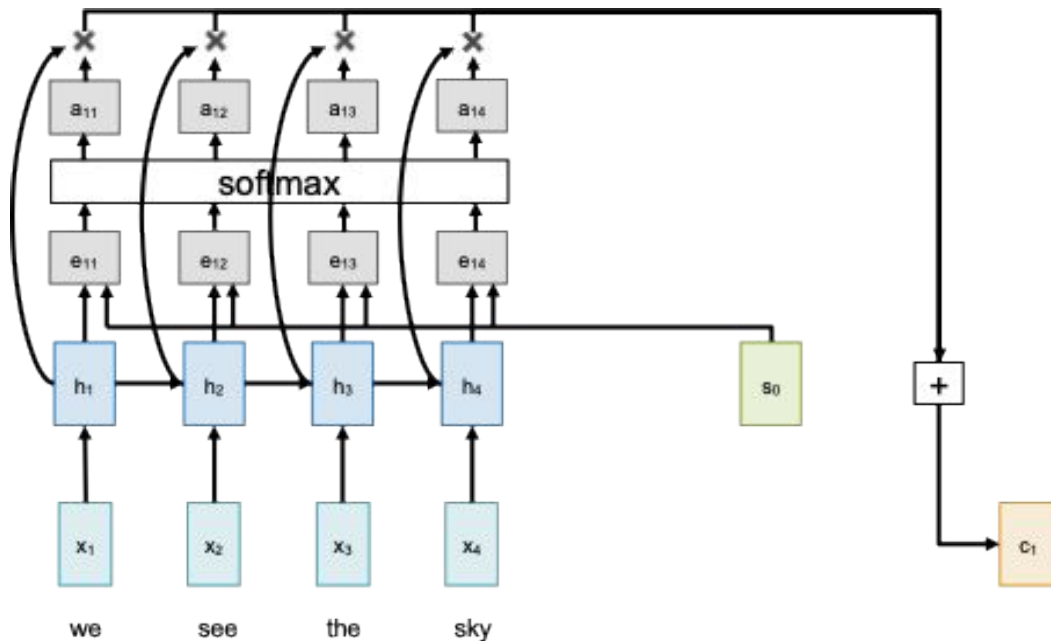
- Use dot product for similarity

Attention Layer

Inputs:

Query vector: $\mathbf{q}$ [D_x]

Data vectors: $\mathbf{X}$ [$N_x \times D_x$]



Computation:

Similarities: e [N_x] $e_i = \mathbf{q} \cdot \mathbf{X}_i / \sqrt{D_x}$

Attention weights: $a = \text{softmax}(e)$ [N_x]

Output vector: $\mathbf{y} = \sum_i a_i \mathbf{X}_i$ [D_x]

Changes

- Use **scaled** dot product for similarity

Attention Layer

Inputs:

Query vector: $\mathbf{q}$ [D_x]

Data vectors: $\mathbf{X}$ [$N_x \times D_x$]

Large similarities will cause softmax to saturate and give vanishing gradients

Recall $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}| \cos(\text{angle})$

Suppose that $\mathbf{a}$ and $\mathbf{b}$ are constant vectors of dimension D

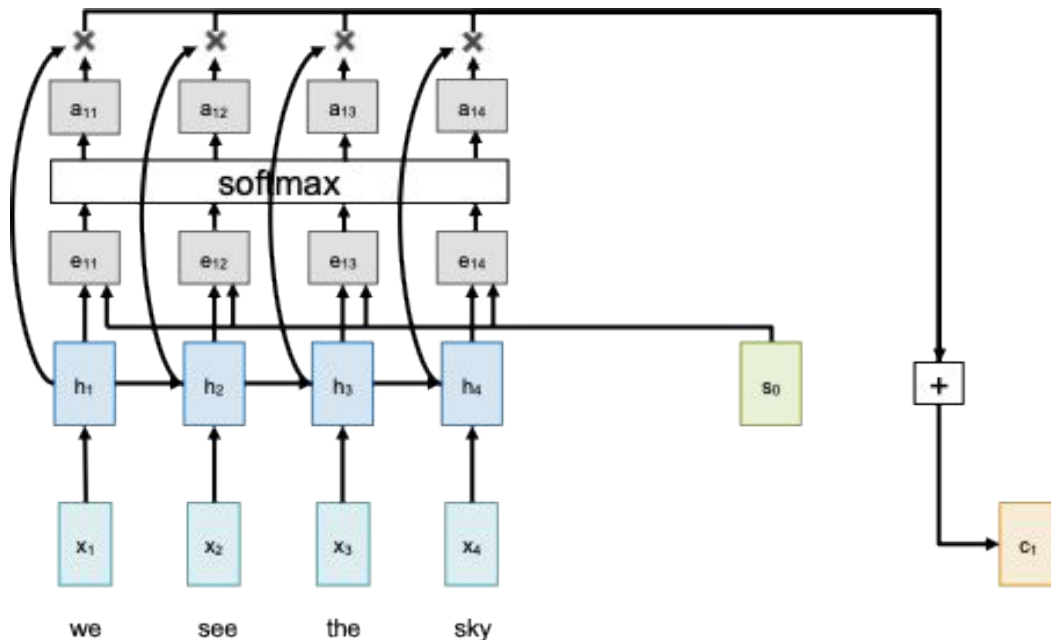
Then $|\mathbf{a}| = (\sum_i a_i^2)^{1/2} = \sqrt{D}$

Computation:

Similarities: $\mathbf{e}$ [N_x] $e_i = \mathbf{q} \cdot \mathbf{X}_i / \sqrt{D_x}$

Attention weights: $\mathbf{a} = \text{softmax}(\mathbf{e})$ [N_x]

Output vector: $\mathbf{y} = \sum_i a_i \mathbf{X}_i$ [D_x]



Changes

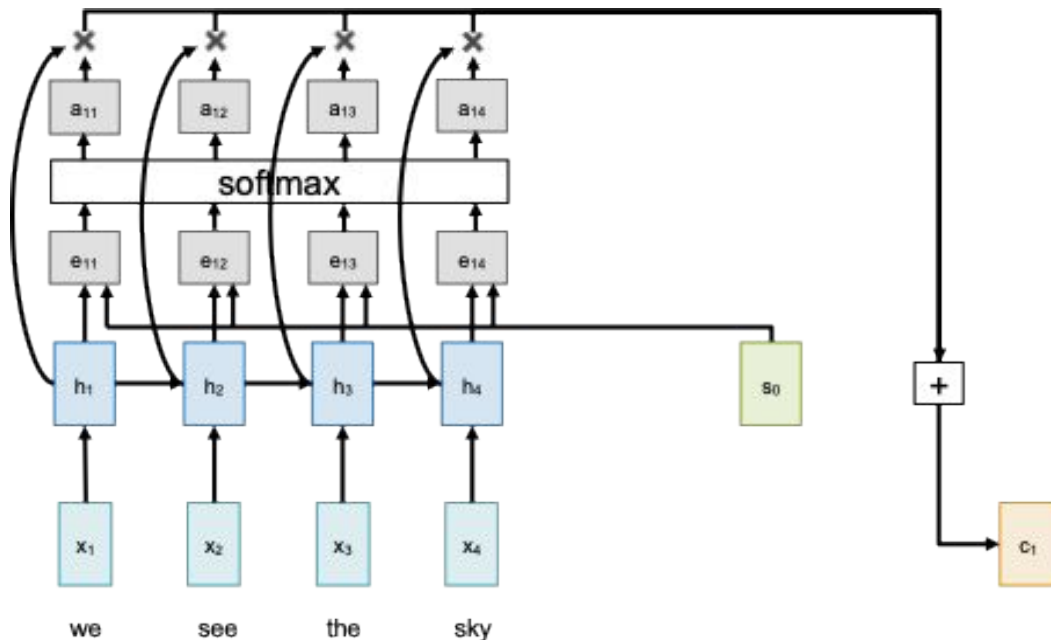
- Use **scaled** dot product for similarity

Attention Layer

Inputs:

Query vector: **Q** [$N_Q \times D_X$]

Data vectors: **X** [$N_X \times D_X$]



Computation:

Similarities: $E = QX^T / \sqrt{D_X} [N_Q \times N_X]$

$$E_{ij} = Q_i \cdot X_j / \sqrt{D_X}$$

Attention weights: $A = \text{softmax}(E, \text{dim}=1) [N_Q \times N_X]$

Output vector: $Y = AX [N_Q \times D_X]$

$$Y_i = \sum_j A_{ij} X_j$$

Changes

- Use scaled dot product for similarity
- Multiple **query** vectors

Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

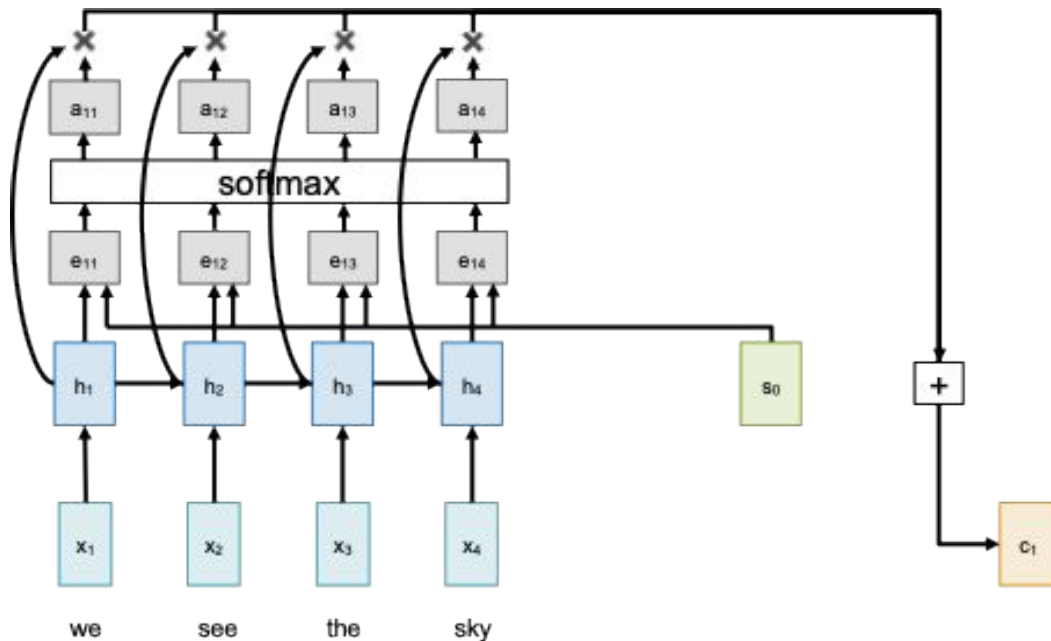
Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]



Changes

- Use scaled dot product for similarity
- Multiple **query** vectors
- Separate **key** and **value**

Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

$\mathbf{X}_1$

$\mathbf{X}_2$

$\mathbf{X}_3$

$\mathbf{Q}_1$

$\mathbf{Q}_2$

$\mathbf{Q}_3$

$\mathbf{Q}_4$

Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

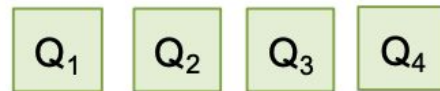
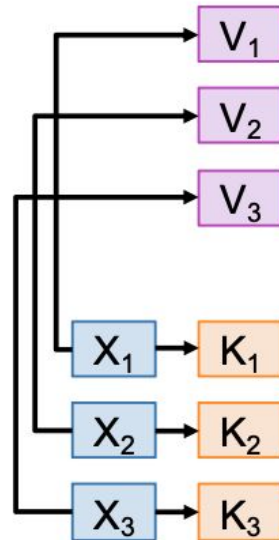
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$



Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

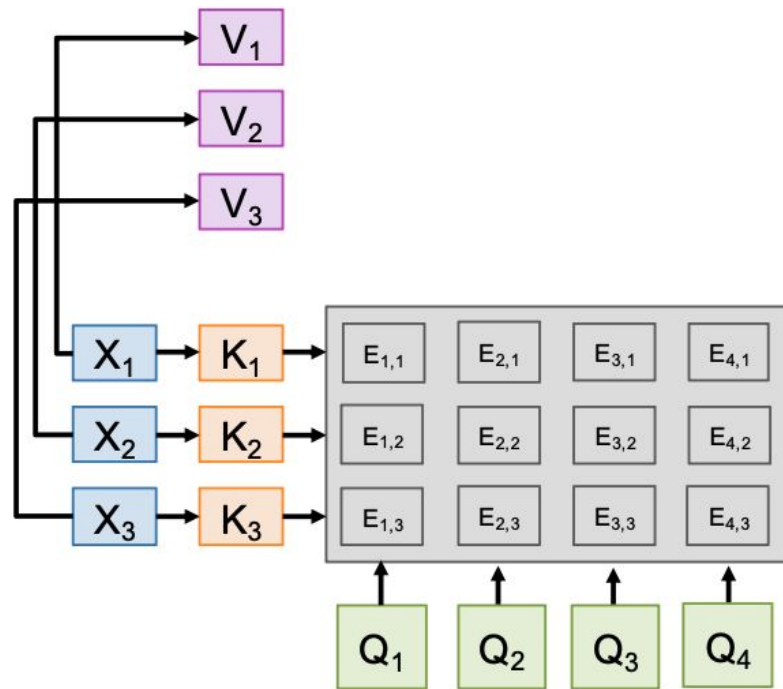
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$



Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

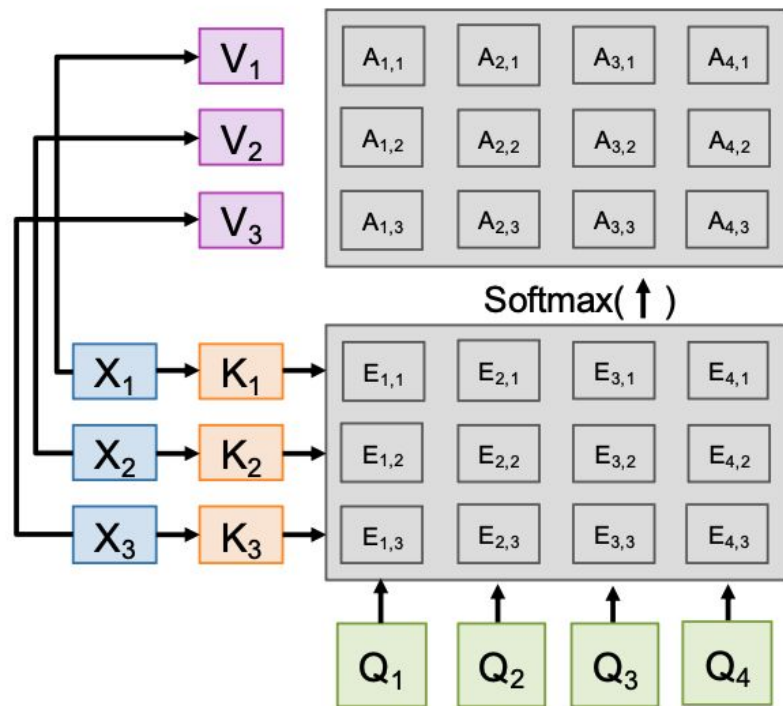
$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]

$$Y_i = \sum_j A_{ij} V_j$$

Softmax normalizes each column: each **query** predicts a distribution over the **keys**



Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

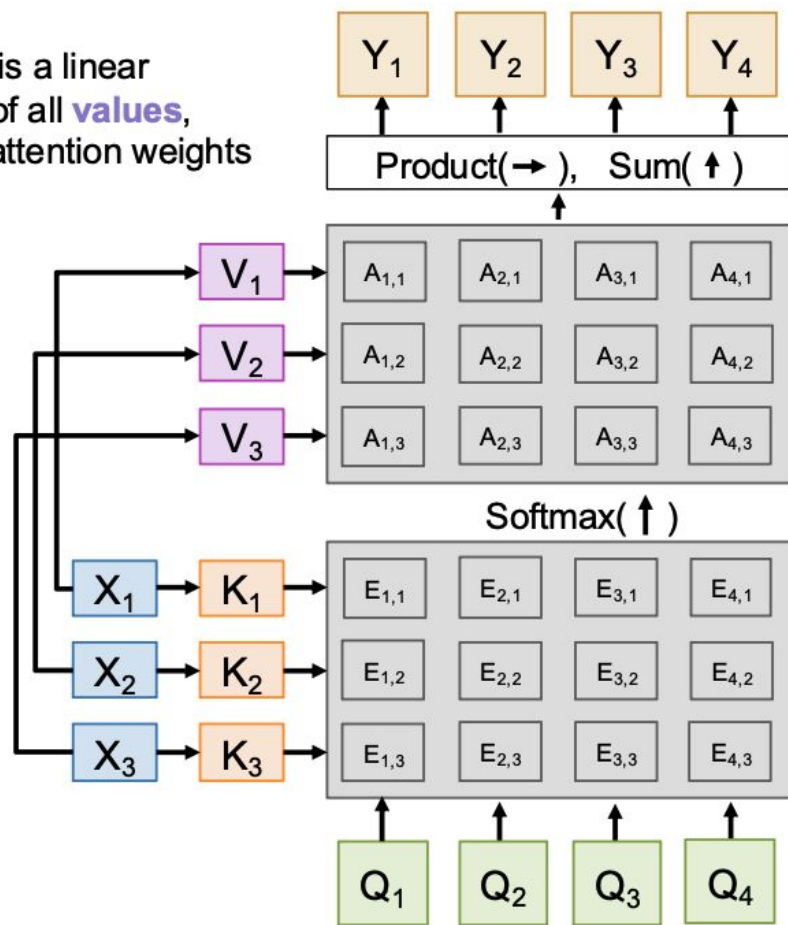
$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]

$$Y_i = \sum_j A_{ij} V_j$$

Each **output** is a linear combination of all **values**, weighted by attention weights



Cross-Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Each **query** produces one **output**, which is a mix of information in the **data** vectors

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

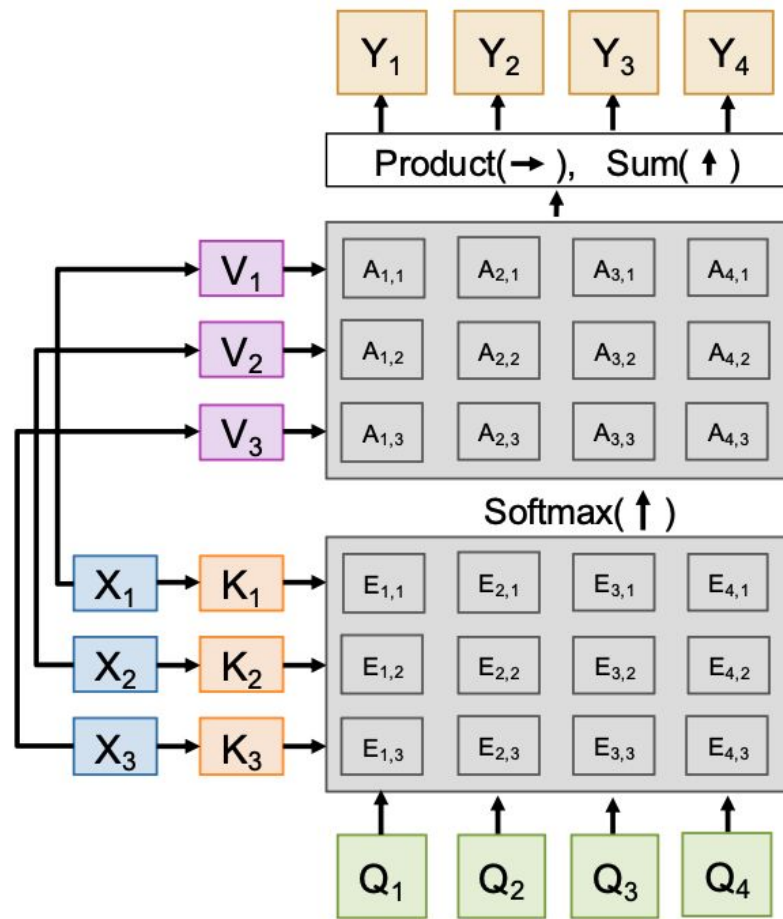
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]

$$Y_i = \sum_j A_{ij} V_j$$



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

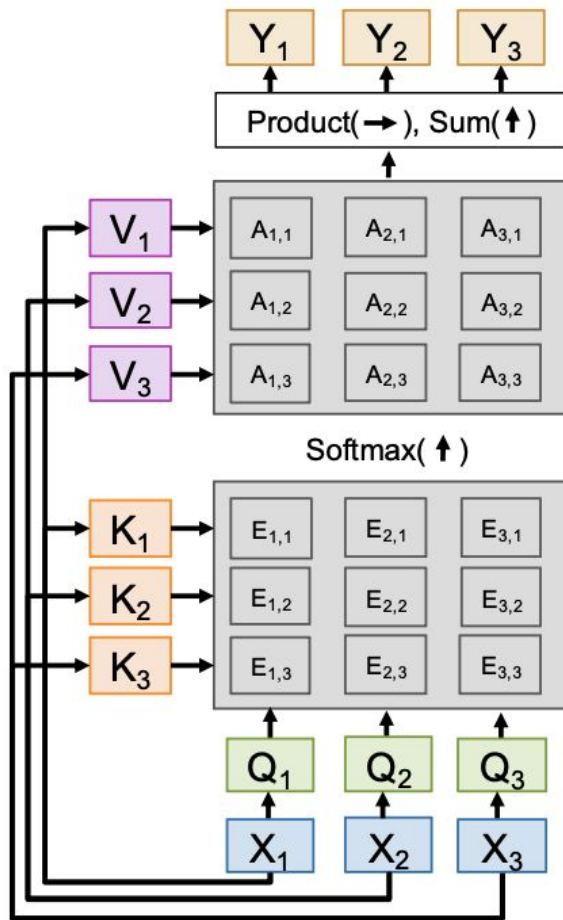
Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

Each **input** produces one **output**, which is a mix of information from all **inputs**

Shapes get a little simpler:

- N input vectors, each D_{in}
- Almost always $D_Q = D_V = D_{out}$



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Each **input** produces one **output**, which is a mix of information from all **inputs**

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

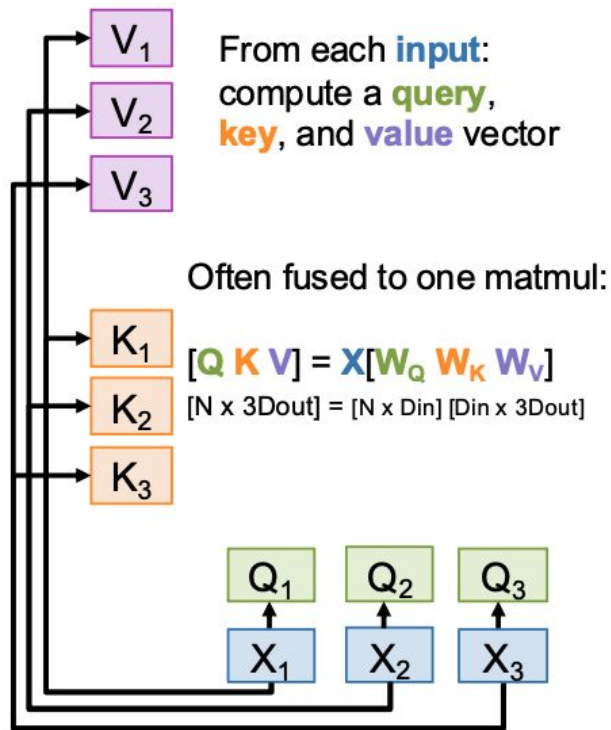
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Each **input** produces one **output**, which is a mix of information from all **inputs**

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

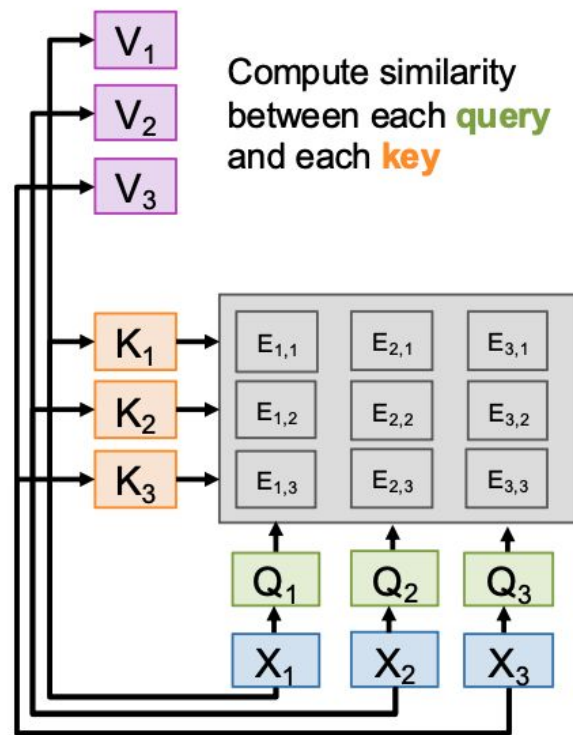
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Each **input** produces one **output**, which is a mix of information from all **inputs**

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

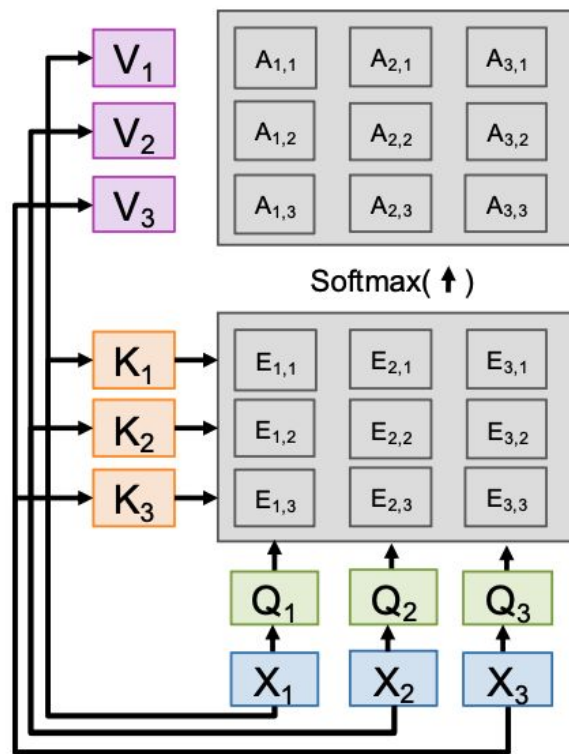
$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

Normalize over each column:
each **query** computes a
distribution over **keys**



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Each **input** produces one **output**, which is a mix of information from all **inputs**

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

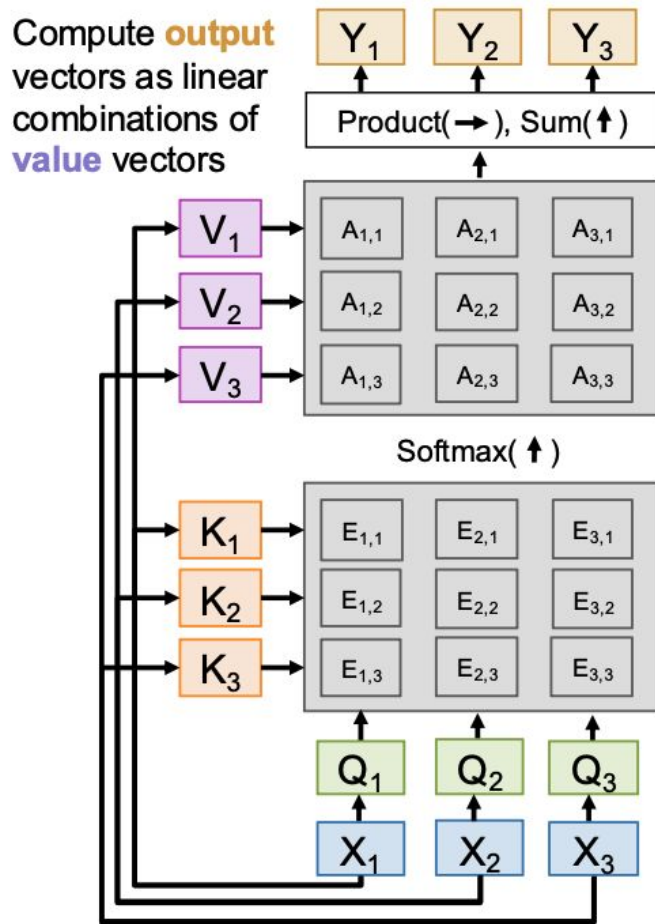
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

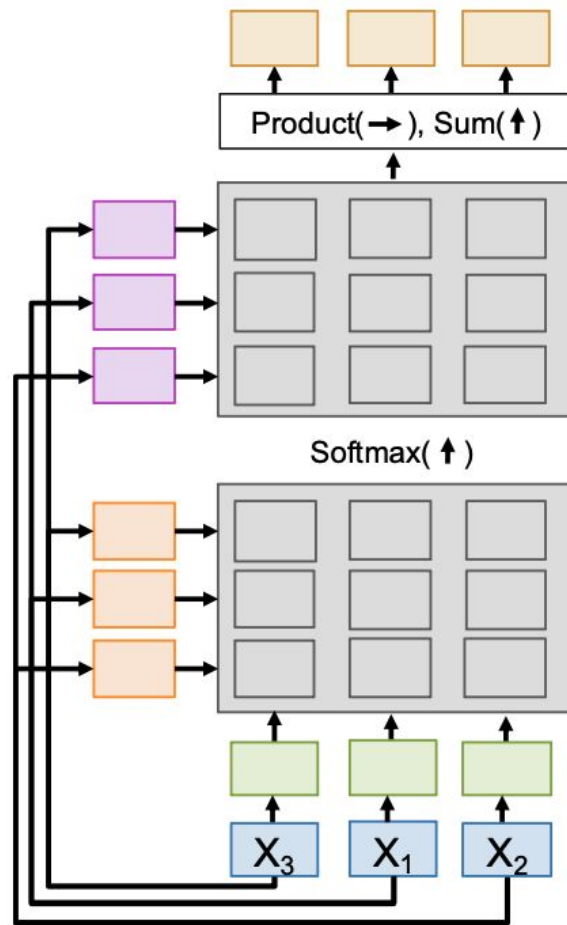
$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

Consider permuting **inputs**:



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

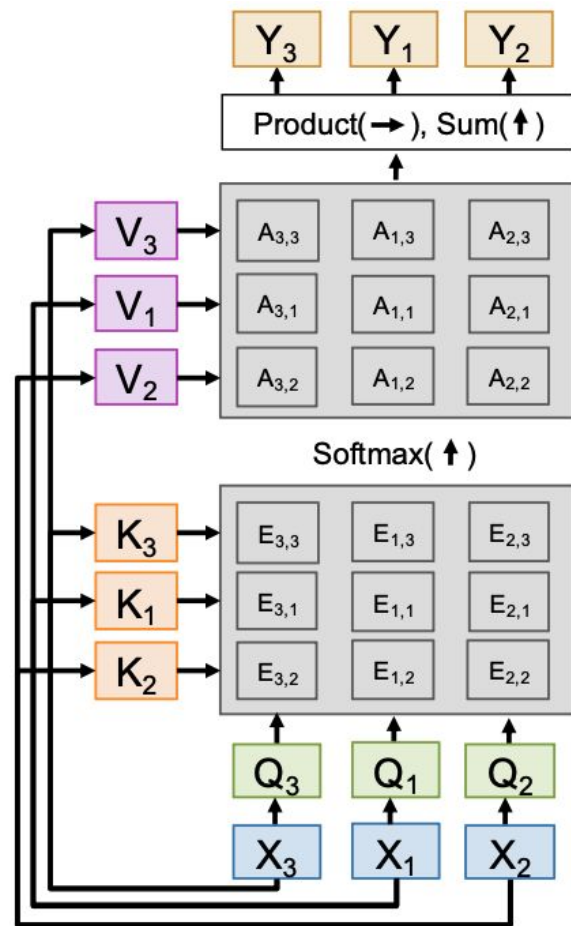
Consider permuting **inputs**:

Queries, **keys**, and **values** will be the same but permuted

Similarities are the same but permuted

Attention weights are the same but permuted

Outputs are the same but permuted



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

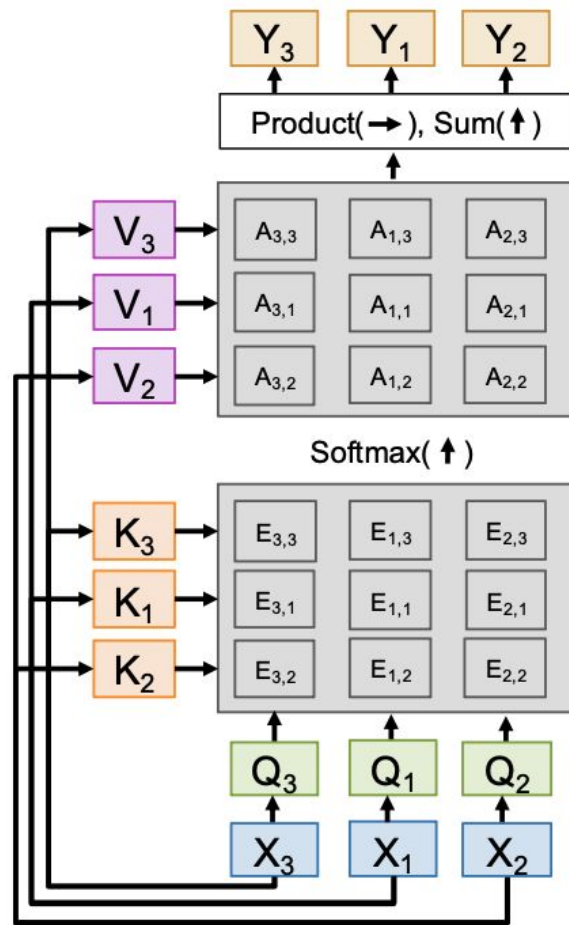
Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

Self-Attention is
permutation equivariant:
 $F(\sigma(X)) = \sigma(F(X))$

This means that Self-Attention
works on **sets of vectors**



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

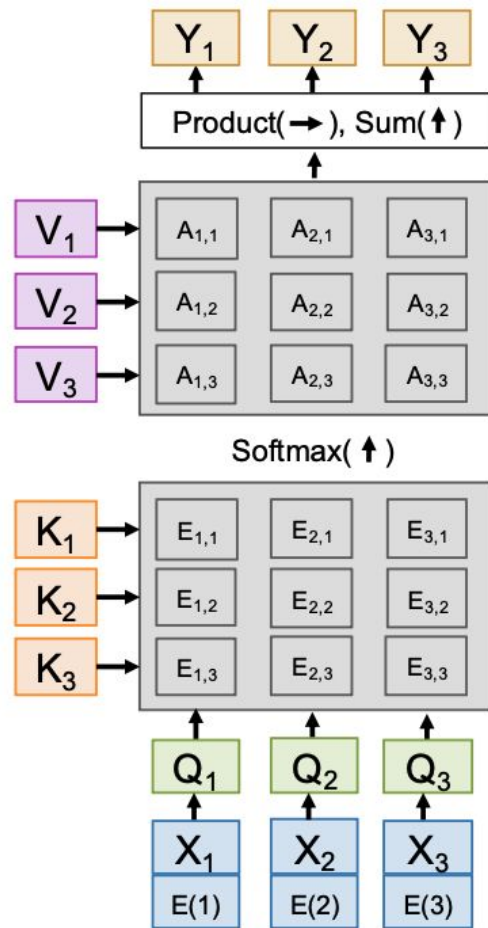
Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

Problem: Self-Attention does not know the order of the sequence

Solution: Add positional encoding to each input; this is a vector that is a fixed function of the index



Masked Self-Attention Layer

Don't let vectors "look ahead" in the sequence

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Override similarities with $-\infty$;
this controls which inputs each
vector is allowed to look at.

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

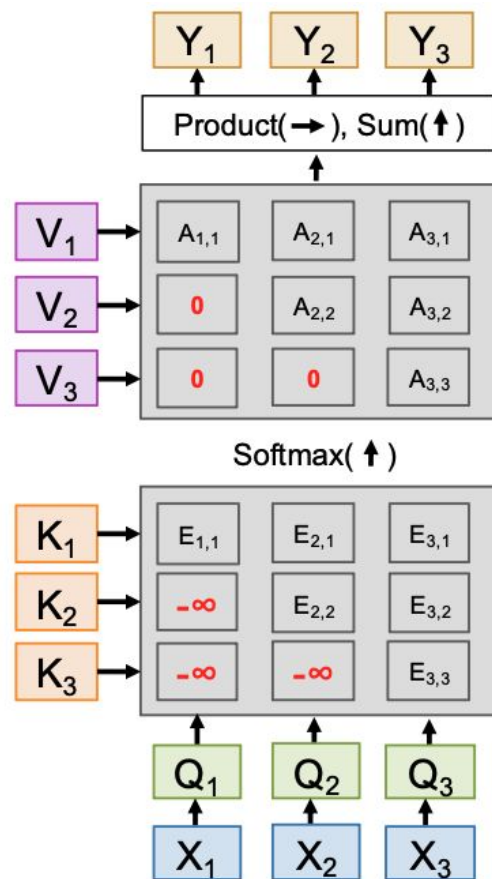
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$



Masked Self-Attention Layer

Don't let vectors "look ahead" in the sequence

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

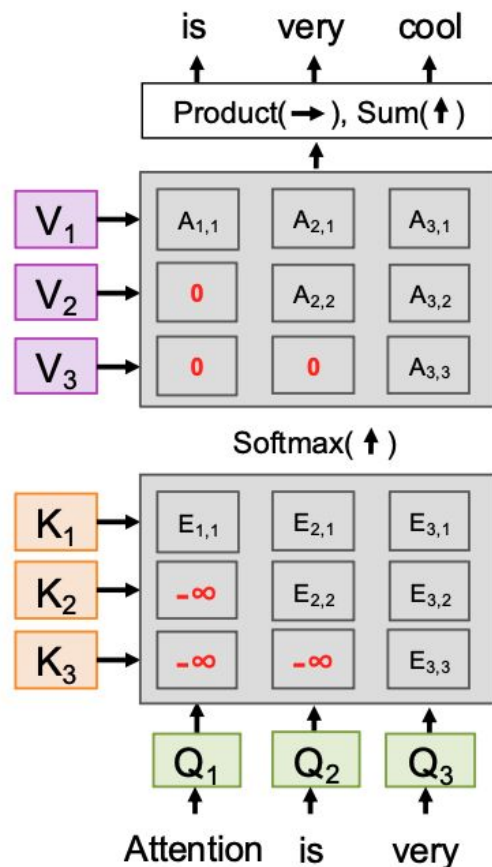
Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

Override similarities with $-\infty$; this controls which inputs each vector is allowed to look at.

Used for language modeling where you want to predict the next word



Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: $X [N \times D_{in}]$

Key matrix: $W_K [D_{in} \times D_{out}]$

Value matrix: $W_V [D_{in} \times D_{out}]$

Query matrix: $W_Q [D_{in} \times D_{out}]$

Computation:

Queries: $Q = XW_Q [N \times D_{out}]$

Keys: $K = XW_K [N \times D_{out}]$

Values: $V = XW_V [N \times D_{out}]$

Similarities: $E = QK^T / \sqrt{D_Q} [N \times N]$

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim}=1) [N \times N]$

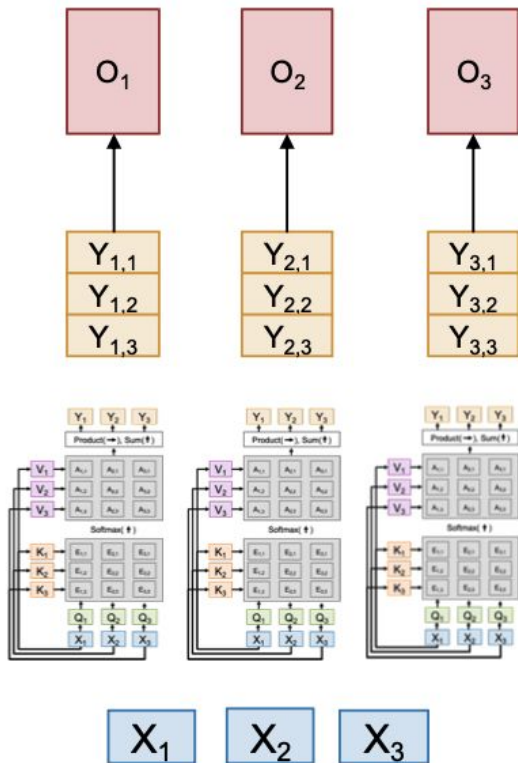
Output vector: $Y = AX [N \times D_{out}]$

$$Y_i = \sum_j A_{ij} V_j$$

Output projection fuses data from each head

Stack up the H independent outputs for each input X

H = 3 independent self-attention layers (called heads), each with their own weights



Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: $\mathbf{X}$ [N x D]

Key matrix: $\mathbf{W}_K$ [D x HD_H]

Value matrix: $\mathbf{W}_V$ [D x HD_H]

Query matrix: $\mathbf{W}_Q$ [D x HD_H]

Output matrix: $\mathbf{W}_O$ [HD_H x D]

Each of the H parallel layers use a qkv dim of D_H = "head dim"

Usually $D_H = D / H$, so inputs and outputs have the same dimension

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [H x N x D_H]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [H x N x D_H]

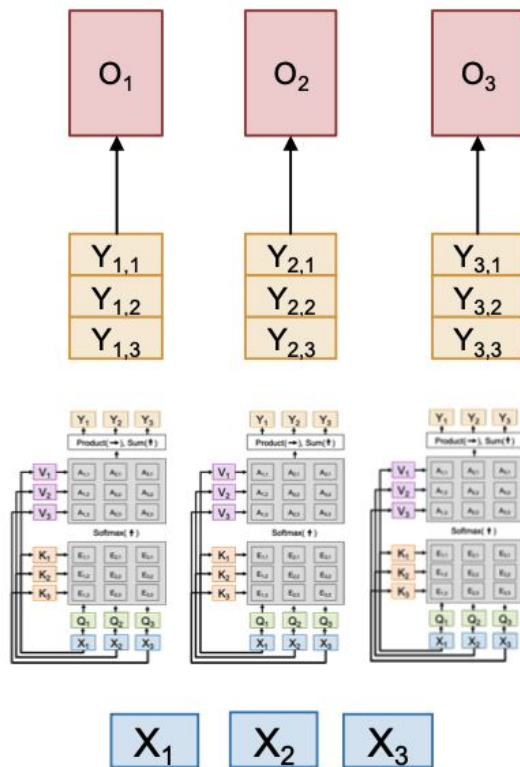
Values: $\mathbf{V} = \mathbf{XW}_V$ [H x N x D_H]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [H x N x N]

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ [H x N x N]

Head outputs: $\mathbf{Y} = \mathbf{AV}$ [H x N x D_H] => [N x HD_H]

Outputs: $\mathbf{O} = \mathbf{YW}_O$ [N x D]



Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D$]

Key matrix: $\mathbf{W}_K$ [$D \times HD_H$]

Value matrix: $\mathbf{W}_V$ [$D \times HD_H$]

Query matrix: $\mathbf{W}_Q$ [$D \times HD_H$]

Output matrix: $\mathbf{W}_O$ [$HD_H \times D$]

In practice, compute all H heads in parallel using batched matrix multiply operations.

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$H \times N \times D_H$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$H \times N \times D_H$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$H \times N \times D_H$]

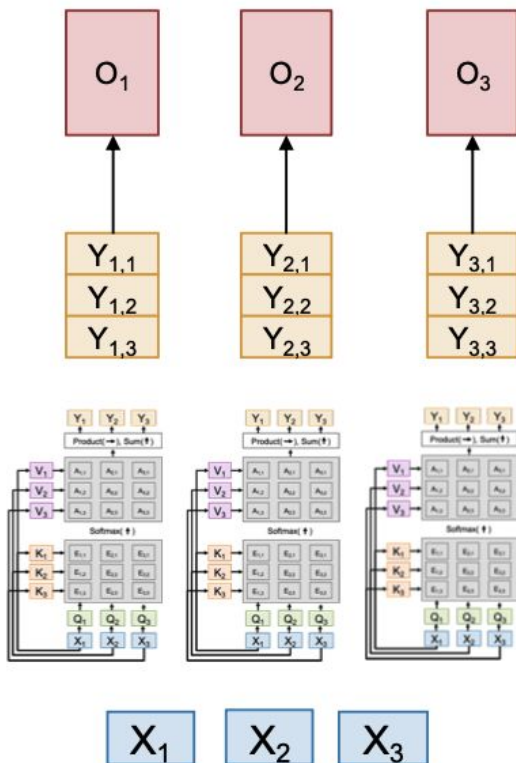
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$H \times N \times N$]

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ [$H \times N \times N$]

Head outputs: $\mathbf{Y} = \mathbf{AV}$ [$H \times N \times D_H$] $\Rightarrow$ [$N \times HD_H$]

Outputs: $\mathbf{O} = \mathbf{YW}_O$ [$N \times D$]

Used everywhere in practice.



Self-Attention is Four Matrix Multiplies!

Inputs:

Input vectors: $\mathbf{X}$ $[N \times D]$

Key matrix: $\mathbf{W}_K$ $[D \times HD_H]$

Value matrix: $\mathbf{W}_V$ $[D \times HD_H]$

Query matrix: $\mathbf{W}_Q$ $[D \times HD_H]$

Output matrix: $\mathbf{W}_O$ $[HD_H \times D]$

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[H \times N \times D_H]$

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[H \times N \times D_H]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ $[H \times N \times N]$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ $[H \times N \times N]$

Head outputs: $\mathbf{Y} = \mathbf{AV}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$

Outputs: $\mathbf{O} = \mathbf{YW}_O$ $[N \times D]$

1. QKV Projection

$[N \times D]$ $[D \times 3HD_H] \Rightarrow [N \times 3HD_H]$

Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

Self-Attention is Four Matrix Multiplies!

Inputs:

Input vectors: $\mathbf{X}$ $[N \times D]$

Key matrix: $\mathbf{W}_K$ $[D \times HD_H]$

Value matrix: $\mathbf{W}_V$ $[D \times HD_H]$

Query matrix: $\mathbf{W}_Q$ $[D \times HD_H]$

Output matrix: $\mathbf{W}_O$ $[HD_H \times D]$

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[H \times N \times D_H]$

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[H \times N \times D_H]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ $[H \times N \times N]$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ $[H \times N \times N]$

Head outputs: $\mathbf{Y} = \mathbf{AV}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$

Outputs: $\mathbf{O} = \mathbf{YW}_O$ $[N \times D]$

1. QKV Projection

$[N \times D] [D \times 3HD_H] \Rightarrow [N \times 3HD_H]$

Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

2. QK Similarity

$[H \times N \times D_H] [H \times D_H \times N] \Rightarrow [H \times N \times N]$

Self-Attention is Four Matrix Multiplies!

Inputs:

Input vectors: $\mathbf{X}$ $[N \times D]$

Key matrix: $\mathbf{W}_K$ $[D \times HD_H]$

Value matrix: $\mathbf{W}_V$ $[D \times HD_H]$

Query matrix: $\mathbf{W}_Q$ $[D \times HD_H]$

Output matrix: $\mathbf{W}_O$ $[HD_H \times D]$

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[H \times N \times D_H]$

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[H \times N \times D_H]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ $[H \times N \times N]$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ $[H \times N \times N]$

Head outputs: $\mathbf{Y} = \mathbf{AV}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$

Outputs: $\mathbf{O} = \mathbf{YW}_O$ $[N \times D]$

1. QKV Projection

$[N \times D]$ $[D \times 3HD_H] \Rightarrow [N \times 3HD_H]$

Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

2. QK Similarity

$[H \times N \times D_H]$ $[H \times D_H \times N] \Rightarrow [H \times N \times N]$

3. V-Weighting

$[H \times N \times N]$ $[H \times N \times D_H] \Rightarrow [H \times N \times D_H]$

Reshape to $[N \times HD_H]$

Self-Attention is Four Matrix Multiplies!

Inputs:

Input vectors: $\mathbf{X}$ $[N \times D]$

Key matrix: $\mathbf{W}_K$ $[D \times HD_H]$

Value matrix: $\mathbf{W}_V$ $[D \times HD_H]$

Query matrix: $\mathbf{W}_Q$ $[D \times HD_H]$

Output matrix: $\mathbf{W}_O$ $[HD_H \times D]$

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[H \times N \times D_H]$

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[H \times N \times D_H]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ $[H \times N \times N]$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ $[H \times N \times N]$

Head outputs: $\mathbf{Y} = \mathbf{AV}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$

Outputs: $\mathbf{O} = \mathbf{YW}_O$ $[N \times D]$

1. QKV Projection

$[N \times D]$ $[D \times 3HD_H] \Rightarrow [N \times 3HD_H]$

Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

2. QK Similarity

$[H \times N \times D_H]$ $[H \times D_H \times N] \Rightarrow [H \times N \times N]$

3. V-Weighting

$[H \times N \times N]$ $[H \times N \times D_H] \Rightarrow [H \times N \times D_H]$

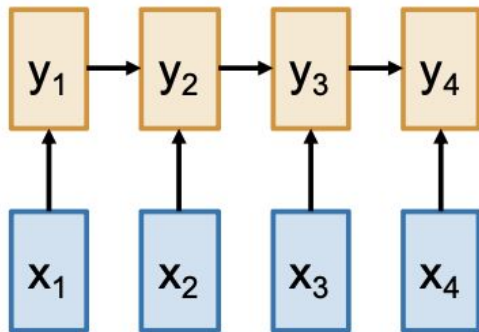
Reshape to $[N \times HD_H]$

4. Output Projection

$[N \times HD_H]$ $[HD_H \times D] \Rightarrow [N \times D]$

Three Ways of Processing Sequences

Recurrent Neural Network



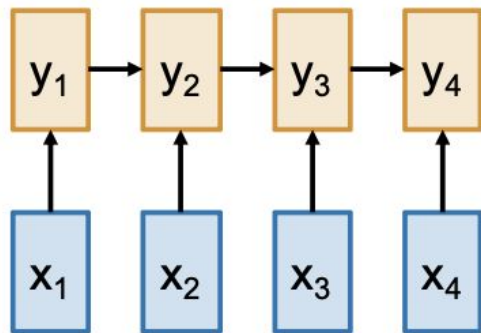
Works on **1D ordered sequences**

(+) Theoretically good at long sequences: $O(N)$ compute and memory for a sequence of length N

(-) Not parallelizable. Need to compute hidden states sequentially

Three Ways of Processing Sequences

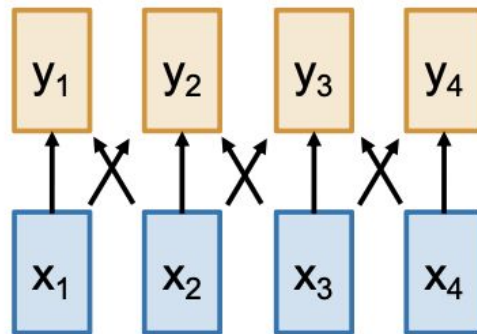
Recurrent Neural Network



Works on **1D ordered sequences**

- (+) Theoretically good at long sequences: $O(N)$ compute and memory for a sequence of length N
- (-) Not parallelizable. Need to compute hidden states sequentially

Convolution

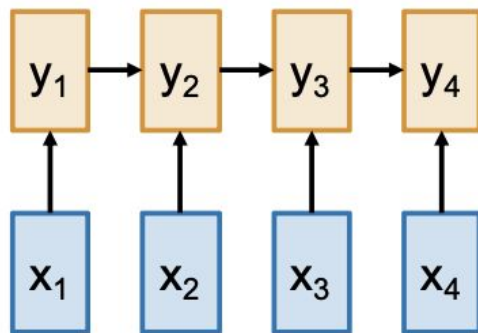


Works on **N-dimensional grids**

- (-) Bad for long sequences: need to stack many layers to build up large receptive fields
- (+) Parallelizable, outputs can be computed in parallel

Three Ways of Processing Sequences

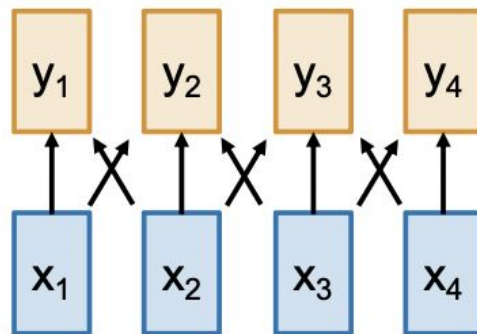
Recurrent Neural Network



Works on **1D ordered sequences**

- (+) Theoretically good at long sequences: $O(N)$ compute and memory for a sequence of length N
- (-) Not parallelizable. Need to compute hidden states sequentially

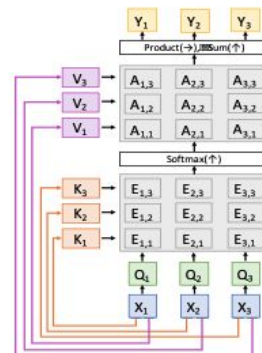
Convolution



Works on **N-dimensional grids**

- (-) Bad for long sequences: need to stack many layers to build up large receptive fields
- (+) Parallelizable, outputs can be computed in parallel

Self-Attention



Works on **sets of vectors**

- (+) Great for long sequences; each output depends directly on all inputs
- (+) Highly parallel, it's just 4 matmuls
- (-) Expensive: $O(N^2)$ compute, $O(N)$ memory for sequence of length N

Three Ways of Processing Sequences

Recurrent Neural Network

Convolution

Self-Attention

y_1 y_2 y_3

Attention is All You Need

Vaswani et al, NeurIPS 2017

sequences. $O(N)$ compute and memory for a sequence of length N
(-) Not parallelizable. Need to compute hidden states sequentially

stack many layers to build up large receptive fields
(+) Parallelizable, outputs can be computed in parallel

output depends directly on all inputs
(+) Highly parallel, it's just 4 matmuls
(-) Expensive: $O(N^2)$ compute, $O(N)$ memory for sequence of length N

The Transformer

Transformer Block

Input: Set of vectors x



Vaswani et al, "Attention is all you need," NeurIPS 2017



Slide credit: Justin Johnson

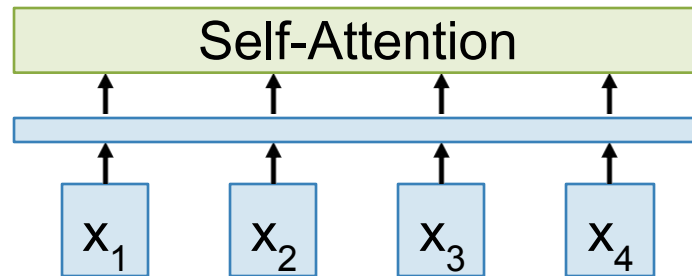
41

The Transformer

Transformer Block

Input: Set of vectors x

All vectors interact through
(multiheaded) Self-Attention

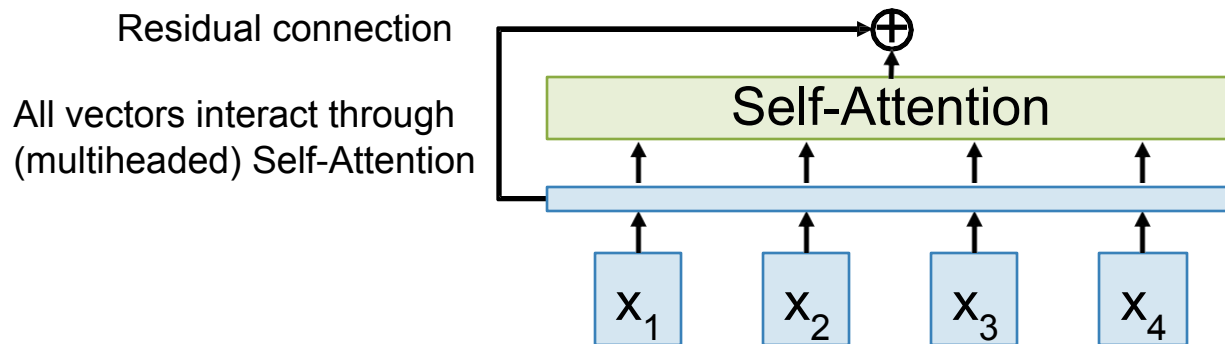


Vaswani et al, "Attention is all you need," NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x



Vaswani et al, "Attention is all you need," NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x

Recall **Layer Normalization**:

Given $h_1, \dots, h_N$ (Shape: D)

scale: γ (Shape: D)

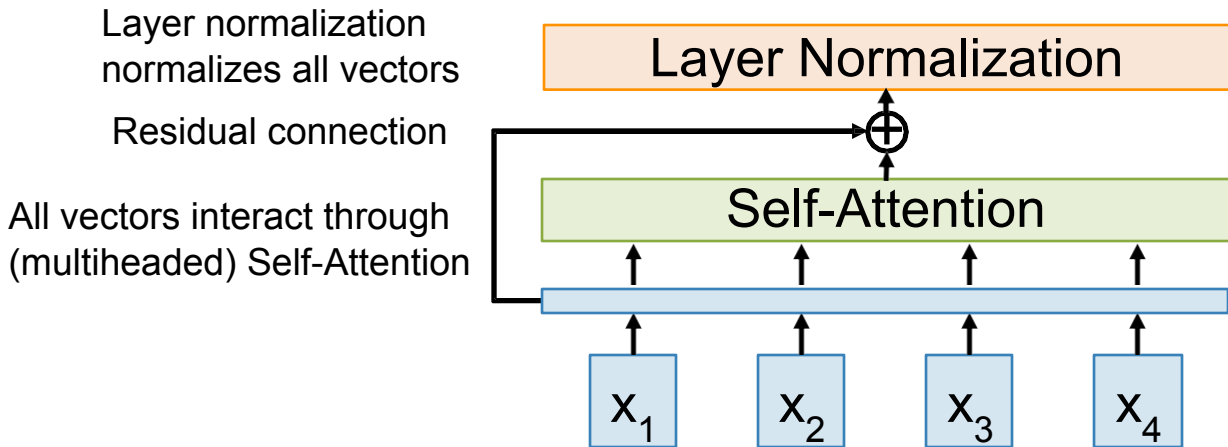
shift: β (Shape: D)

$\mu_i = (\sum_j h_{i,j})/D$ (scalar)

$\sigma_i = (\sum_j (h_{i,j} - \mu_i)^2/D)^{1/2}$ (scalar)

$z_i = (h_i - \mu_i) / \sigma_i$

$y_j = \gamma * z_i + \beta$
Ba et al, 2016

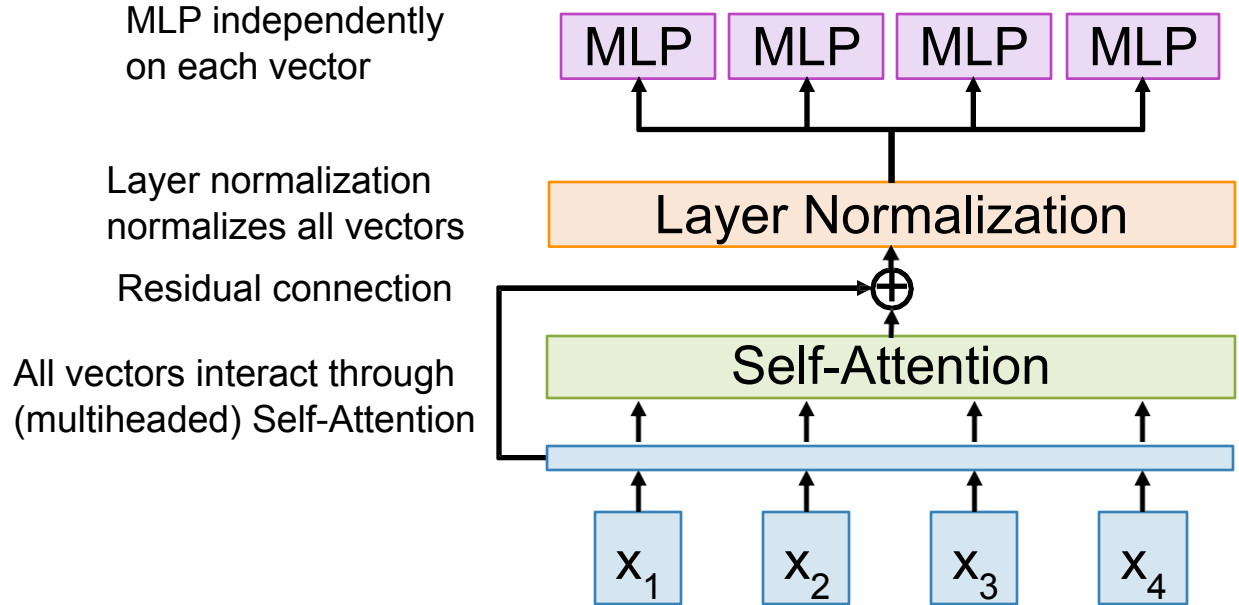


Vaswani et al, "Attention is all you need," NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x



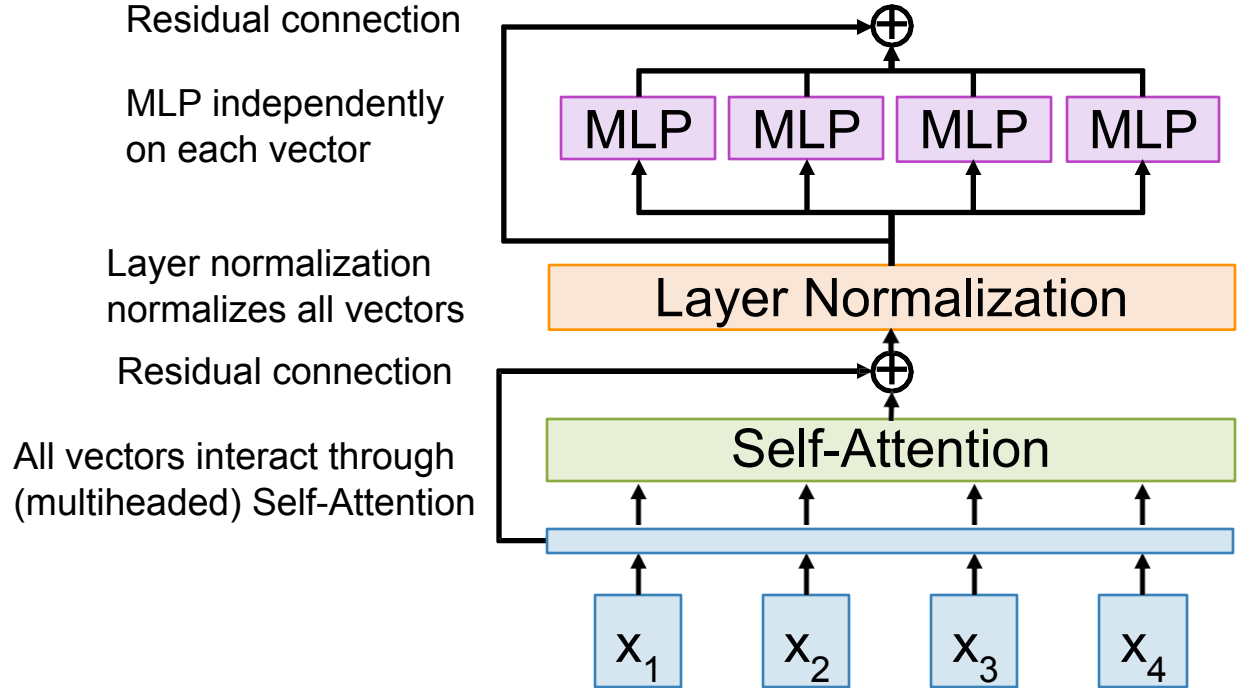
Usually a two-layer MLP;
classic setup is
 $D \Rightarrow 4D \Rightarrow D$

Also sometimes called FFN
(Feed-Forward Network)

The Transformer

Transformer Block

Input: Set of vectors x



Vaswani et al, "Attention is all you need," NeurIPS 2017

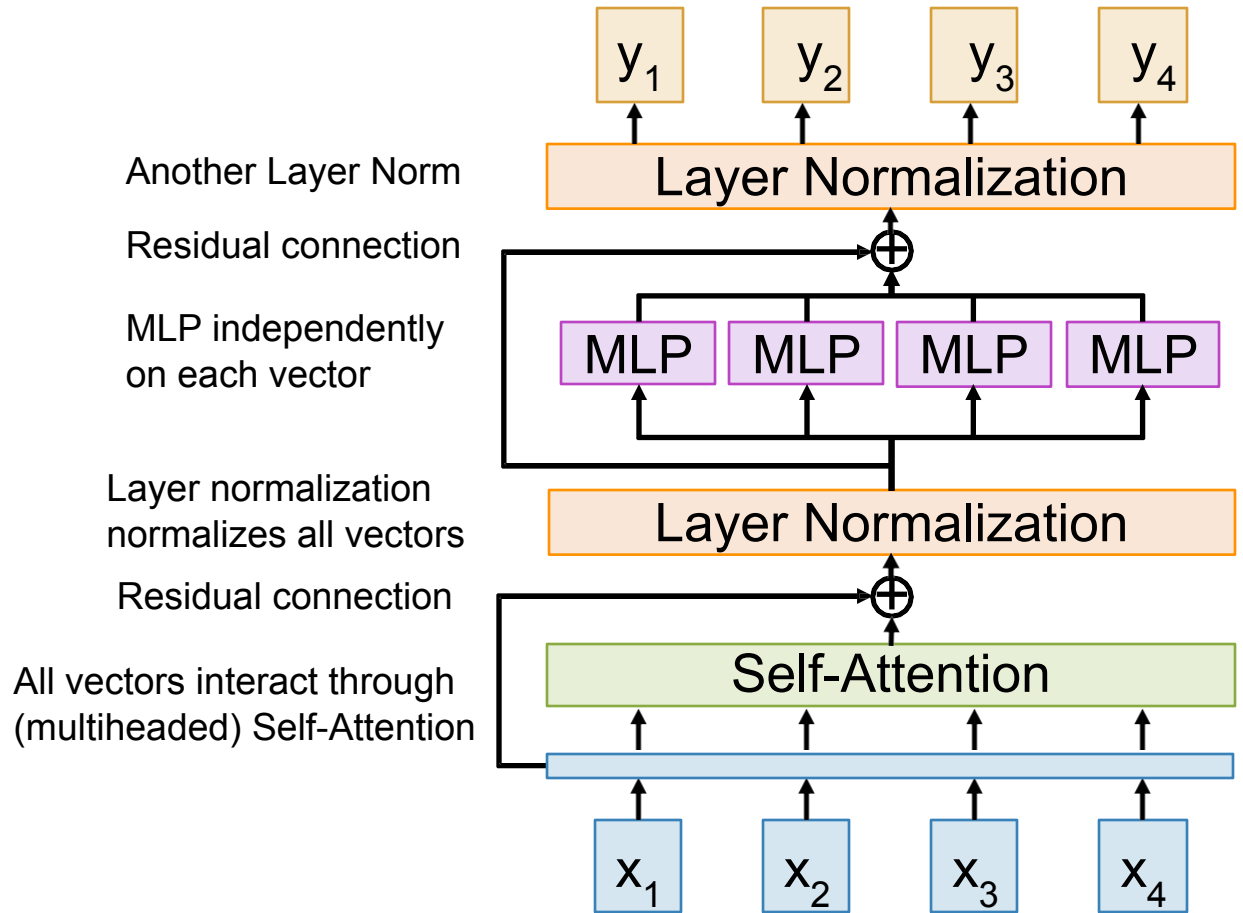


Slide credit: Justin Johnson

The Transformer

Transformer Block

Input: Set of vectors x



Vaswani et al, "Attention is all you need," NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x

Output: Set of vectors y

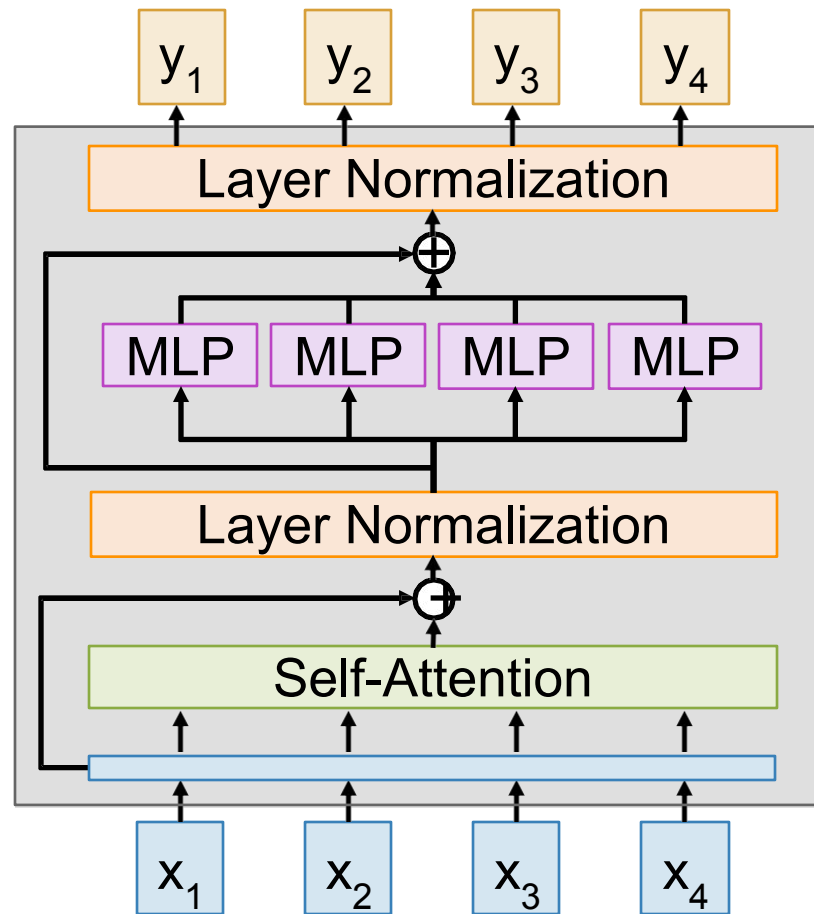
Self-Attention is the only interaction between vectors

LayerNorm and MLP work on each vector independently

Highly scalable and parallelizable, most of the compute is just 6 matmuls:

4 from Self-Attention

2 from MLP



The Transformer

Transformer Block

Input: Set of vectors x

Output: Set of vectors y

Self-Attention is the only interaction between vectors

LayerNorm and MLP work on each vector independently

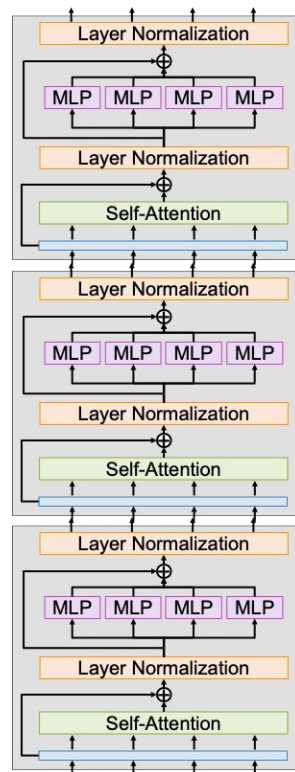
Highly scalable and parallelizable, most of the compute is just 6 matmuls:
4 from Self-Attention
2 from MLP

A **Transformer** is just a stack of identical Transformer blocks!

They have not changed much since 2017... but have gotten a lot bigger

Original: [Vaswani et al, 2017]

12 blocks, $D=1024$, $H=16$, $N=512$
213M params



Vaswani et al, "Attention is all you need," NeurIPS 2017

The Transformer

Transformer Block

Input: Set of vectors x

Output: Set of vectors y

Self-Attention is the only interaction between vectors

LayerNorm and MLP work on each vector independently

Highly scalable and parallelizable, most of the compute is just 6 matmuls:

4 from Self-Attention
2 from MLP

A **Transformer** is just a stack of identical Transformer blocks!

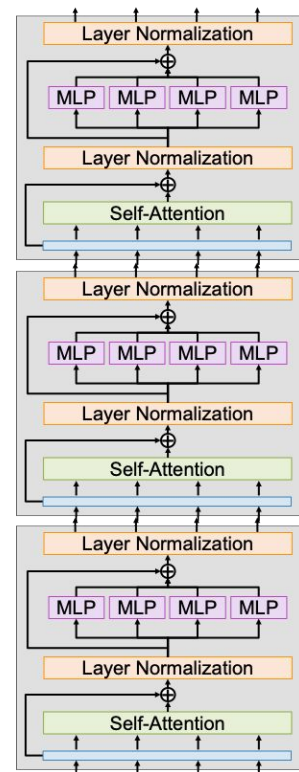
They have not changed much since 2017... but have gotten a lot bigger

Original: [Vaswani et al, 2017]

12 blocks, $D=1024$, $H=16$, $N=512$
213M params

GPT-2: [Radford et al, 2019]

48 blocks, $D=1600$, $H=25$, $N=1024$
1.5B params



The Transformer

Transformer Block

Input: Set of vectors x

Output: Set of vectors y

Self-Attention is the only interaction between vectors

LayerNorm and MLP work on each vector independently

Highly scalable and parallelizable, most of the compute is just 6 matmuls:

- 4 from Self-Attention
- 2 from MLP

A **Transformer** is just a stack of identical Transformer blocks!

They have not changed much since 2017... but have gotten a lot bigger

Original: [Vaswani et al, 2017]

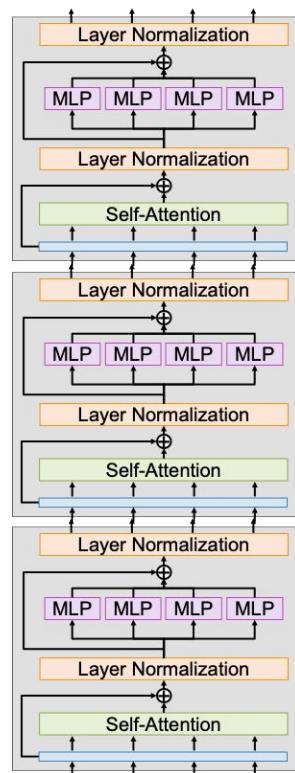
12 blocks, $D=1024$, $H=16$, $N=512$
213M params

GPT-2: [Radford et al, 2019]

48 blocks, $D=1600$, $H=25$, $N=1024$
1.5B params

GPT-3: [Brown et al, 2020]

96 blocks, $D=12288$, $H=96$, $N=2048$
175B params



Transformers for Language Modeling (LLM)

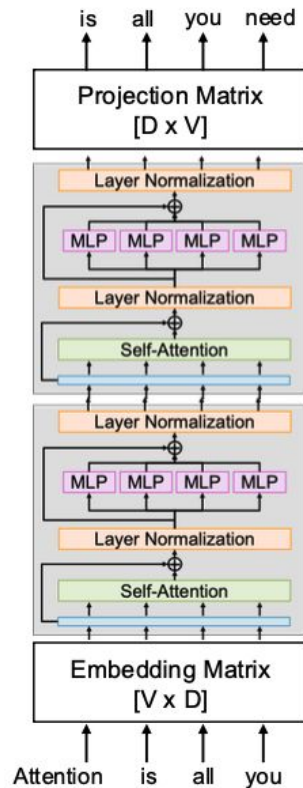
Learn an embedding matrix at the start of the model to convert words into vectors.

Given vocab size V and model dimension D , it's a lookup table of shape $[V \times D]$

Use masked attention inside each transformer block so each token can only see the ones before it

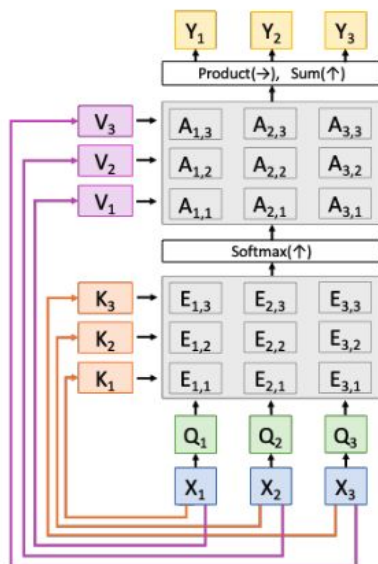
At the end, learn a projection matrix of shape $[D \times V]$ to project each D -dim vector to a V -dim vector of scores for each element of the vocabulary.

Train to predict next token using softmax + cross-entropy loss



Summary: Attention + Transformers

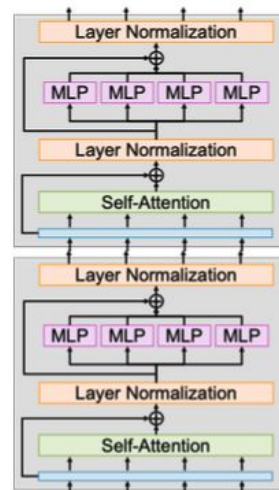
Attention: A new primitive that operates on sets of vectors



Transformers are the backbone of all large AI models today!

Used for language, vision, speech, ...

Transformer: A neural network architecture that uses attention everywhere



Overview

- Unsupervised & Self-Supervised Learning
- Generative Models

Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a *function* to map $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Unsupervised Learning

Data: x

Just data, no labels!

Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering, dimensionality reduction, feature learning, density estimation, etc.

Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a *function* to map $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Unsupervised Learning

Training data is cheap

Data: x

Just data, no labels!

Holy grail: Solve
unsupervised learning:
understand structure
of visual world

Goal: Learn some underlying
hidden *structure* of the data

Examples: Clustering, dimensionality reduction, feature learning, density estimation, etc.

Self-Supervision

1. Expense of producing a new dataset for each new task
2. Some areas are supervision-starved, e.g. medical data, where it is hard to obtain annotation
3. Untapped/availability of vast numbers of unlabelled images/videos
 - Facebook: one billion images uploaded per day
 - 300 hours of video are uploaded to YouTube every minute
4. How infants may learn ...

Self-Supervision

- A form of unsupervised learning where the data provides the supervision
- In general, withhold some part of the data, and task the network with predicting it
- The task defines a proxy loss, and the network is forced to learn what we really care about, e.g. a semantic representation, in order to solve it

Standard approach:

- (1) use self-supervision as a pretext to learn features
- (2) transfer the features to a task with a limited amount of samples

Self-Supervision

Pretext tasks from image transformations

- Rotation, inpainting, rearrangement, coloring
- Reconstruction-based learning (MAE)

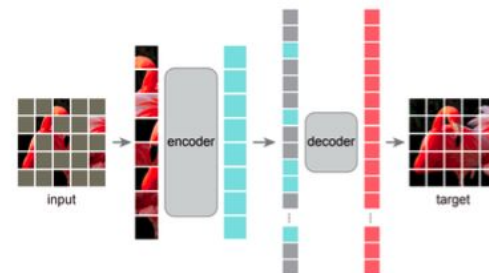


Rotation

Example:



Rearrangement

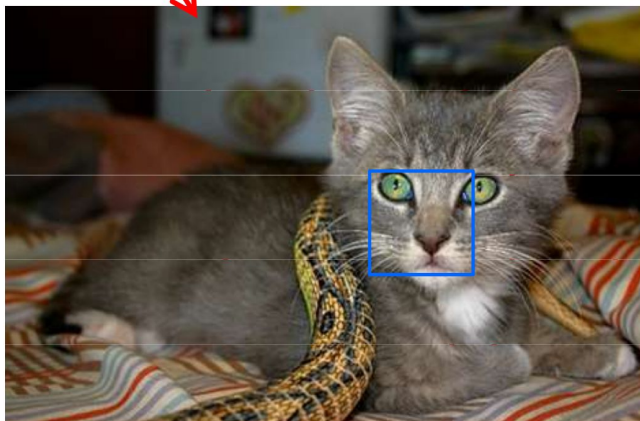
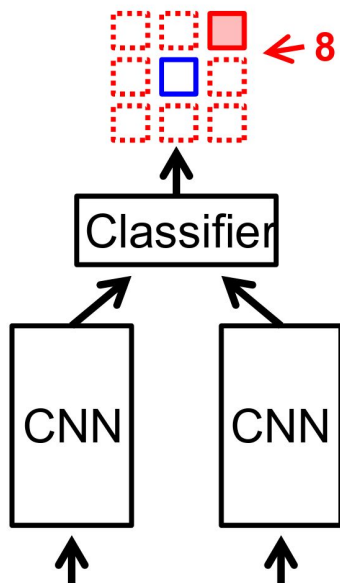


Reconstruction

Self-Supervision

Example: relative positioning

Train network to predict relative position of two regions in the same image



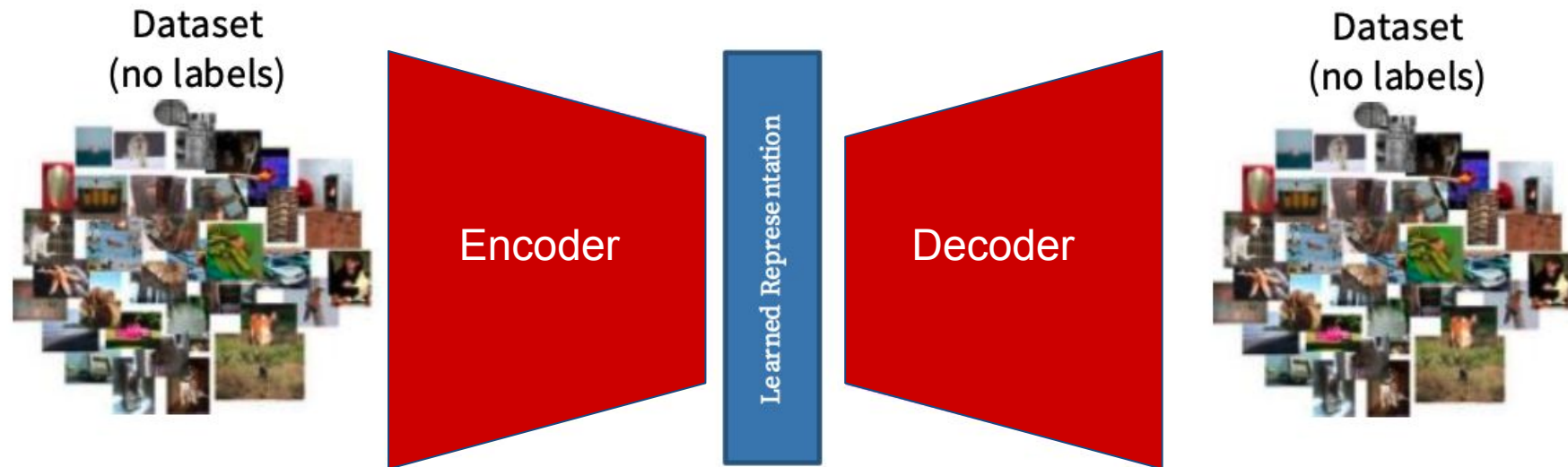
Randomly Sample Patch
Sample Second Patch

Unsupervised visual representation learning by context prediction,
Carl Doersch, Abhinav Gupta, Alexei A. Efros, ICCV 2015

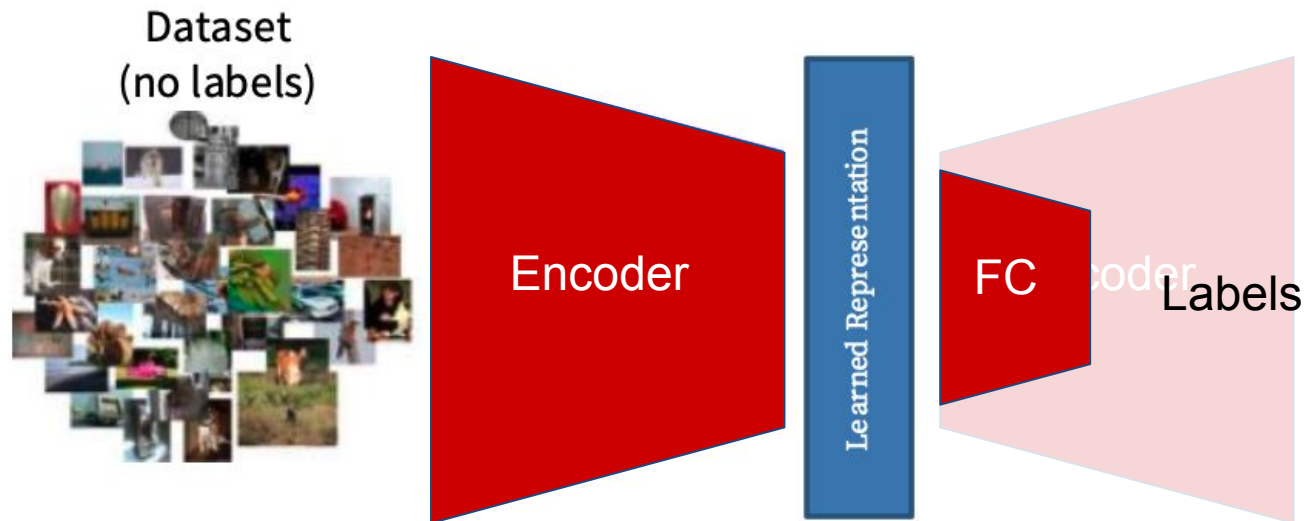
Self-Supervision

Contrastive Learning

Self-Supervision



Self-Supervision



Generative vs Discriminative Models

Discriminative Model:

Learn a probability distribution $p(y|x)$



Assign labels to data
Feature learning (with labels)

Generative Model:

Learn a probability distribution $p(x)$



Detect outliers
Feature learning (without labels)
Sample to generate new data

Conditional Generative Model: Learn $p(x|y)$



Assign labels while rejecting outliers
Sample to generate data from labels

Why Generative Models

Modeling ambiguity: If there are many possible outputs x for an input y , we want to model $P(x | y)$

Language Modeling: Produce output text x from input text y

*Write me a short
rhyming poem about
generative models*



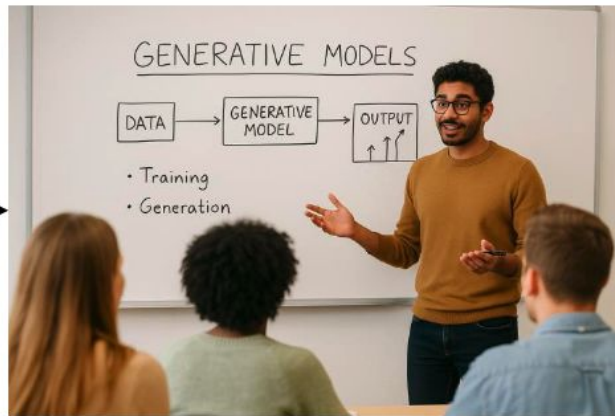
*They sample from a learned P ,
A distribution—structured, free.
Each token comes conditionally,
On all the ones that used to be.*

Why Generative Models

Modeling ambiguity: If there are many possible outputs x for an input y , we want to model $P(x | y)$

Text to Image: Produce output image x from input text y

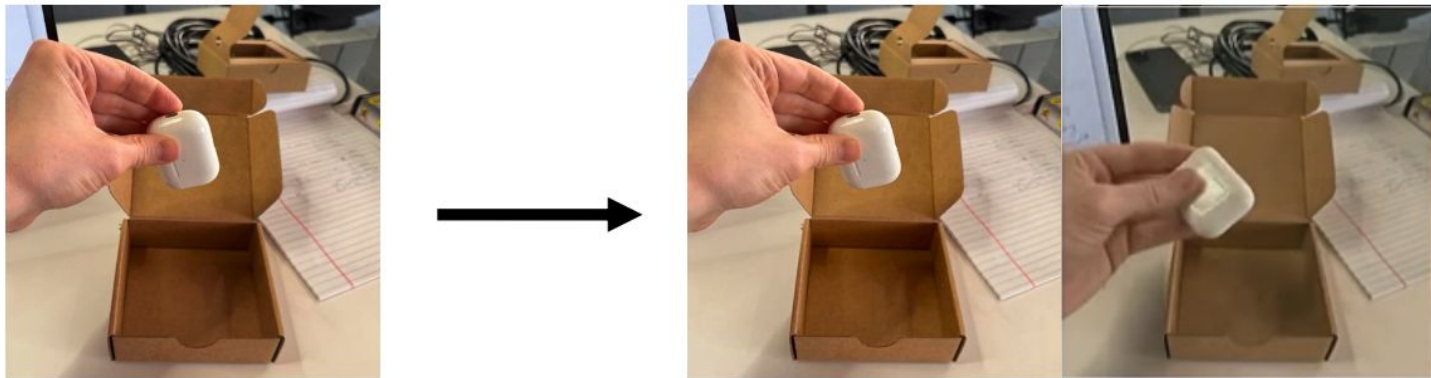
Make me an image showing a person teaching a class on generative models in front of a whiteboard



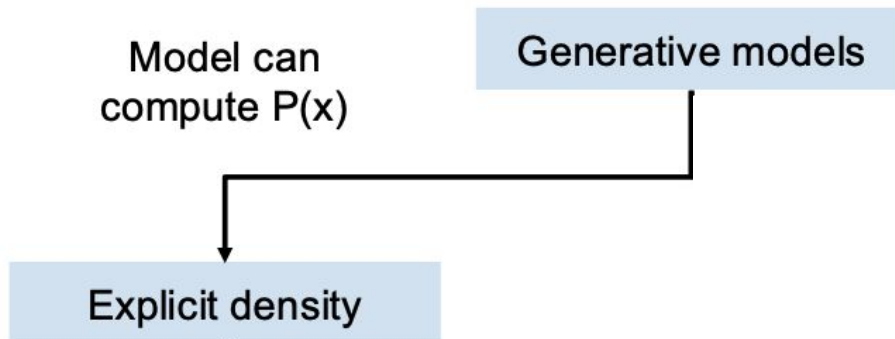
Why Generative Models

Modeling ambiguity: If there are many possible outputs x for an input y , we want to model $P(x | y)$

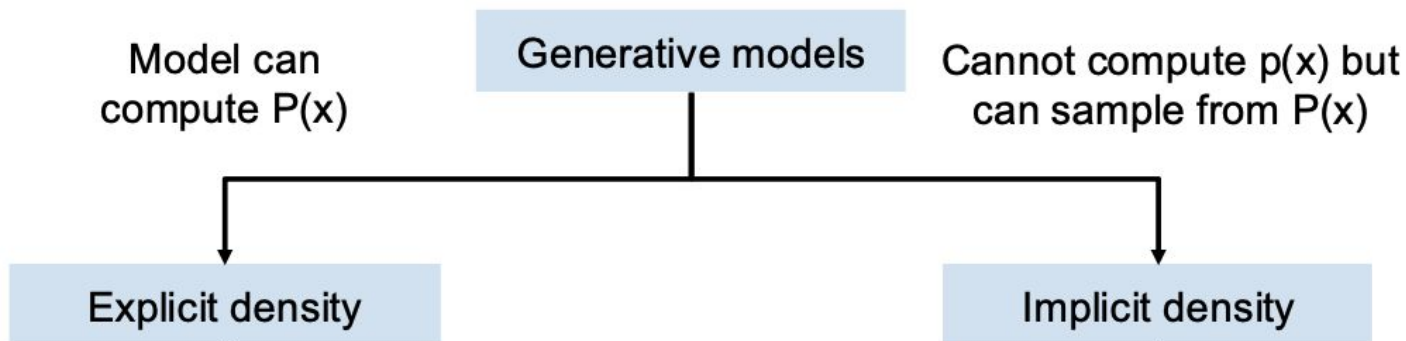
Image to Video: What happens next?



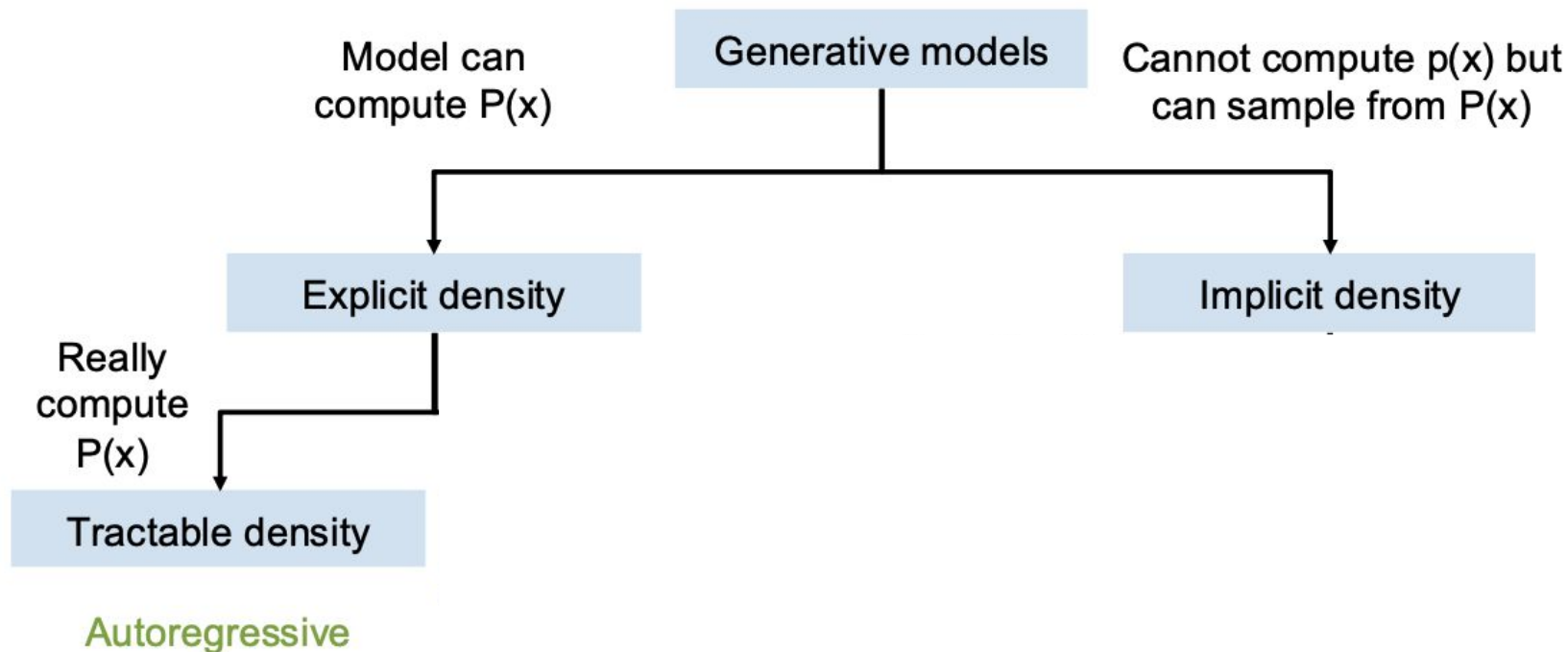
Taxonomy of Generative Models



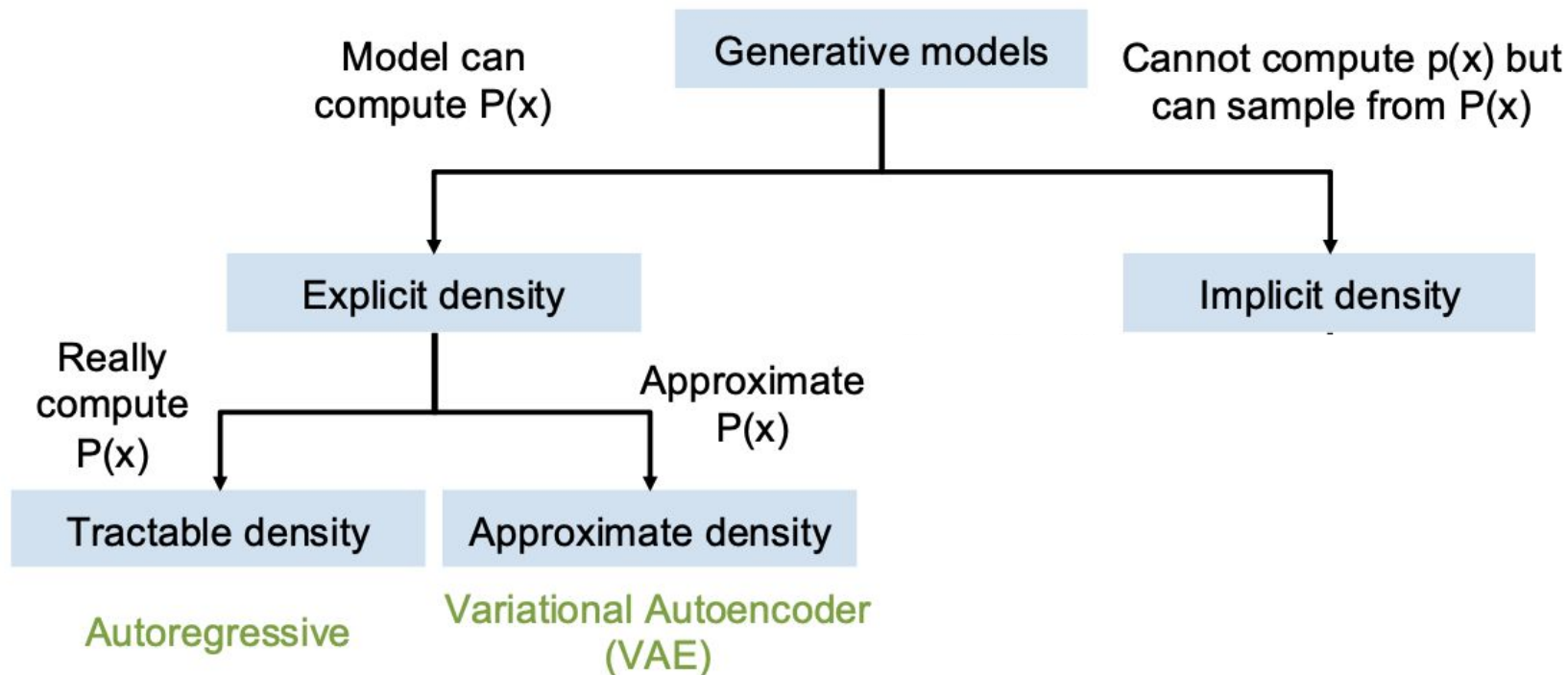
Taxonomy of Generative Models



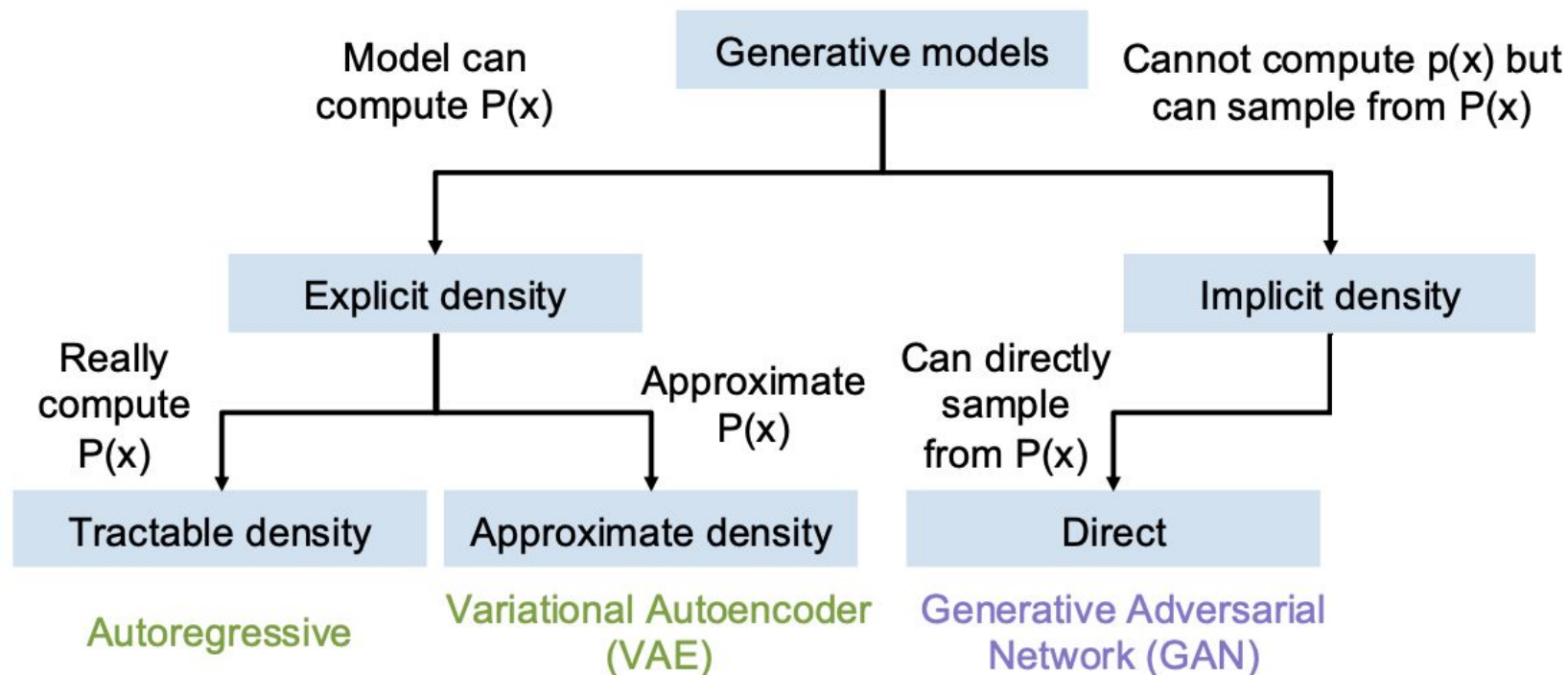
Taxonomy of Generative Models



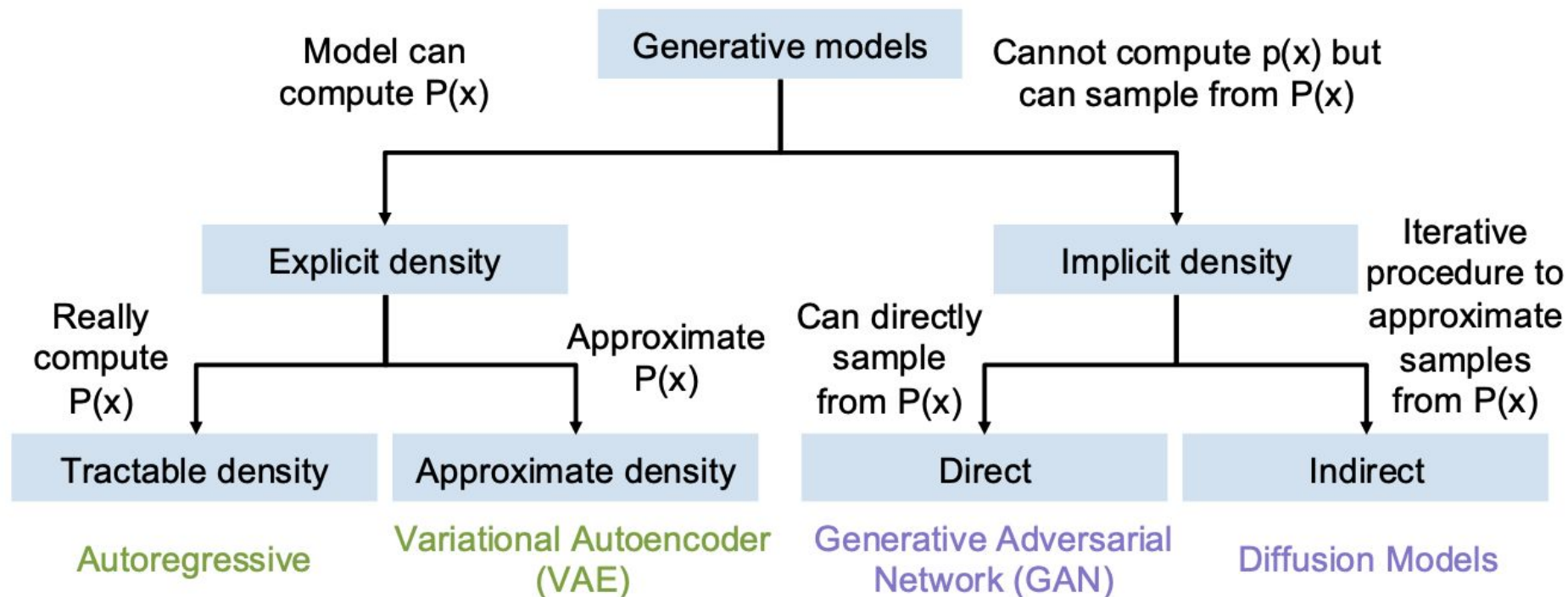
Taxonomy of Generative Models



Taxonomy of Generative Models



Taxonomy of Generative Models



Explicit Density Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

Given dataset $x^{(1)}, x^{(2)}, \dots, x^{(N)}$, train the model by solving:

$$W^* = \arg \max_W \prod_i p(x^{(i)})$$

Maximize probability of training data
(Maximum likelihood estimation)

$$= \arg \max_W \sum_i \log p(x^{(i)})$$

Log trick to exchange product for sum

$$= \arg \max_W \sum_i \log f(x^{(i)}, W)$$

This will be our loss function!
Train with gradient descent

Explicit Density Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

Assume x consists of
multiple subparts:

$$x = (x_1, x_2, x_3, \dots, x_T)$$

Break down probability
using the chain rule:

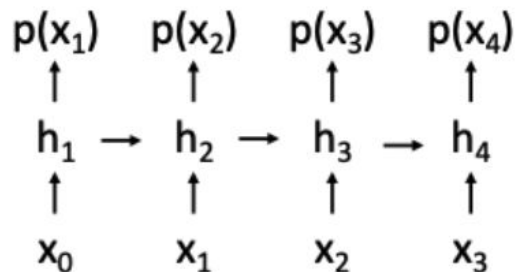
$$\begin{aligned} p(x) &= p(x_1, x_2, x_3, \dots, x_T) \\ &= p(x_1)p(x_2 | x_1)p(x_3 | x_1, x_2) \dots \\ &= \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1}) \end{aligned}$$

Probability of the next subpart
given all the previous subparts

Explicit Density Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

We have already seen this!



Language modeling with RNN

$$x = (x_1, x_2, x_3, \dots, x_T)$$

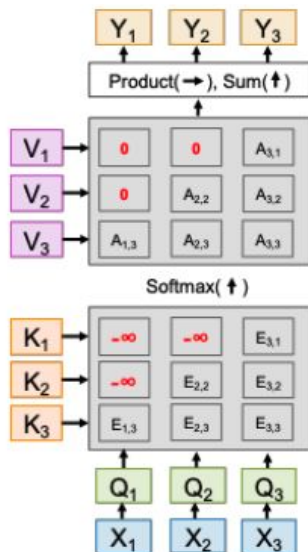
$$\begin{aligned} p(x) &= p(x_1, x_2, x_3, \dots, x_T) \\ &= p(x_1)p(x_2 | x_1)p(x_3 | x_1, x_2) \dots \\ &= \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1}) \end{aligned}$$

Probability of the next subpart
given all the previous subparts

Explicit Density Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

Language
modeling
with masked
Transformer



$$x = (x_1, x_2, x_3, \dots, x_T)$$

$$\begin{aligned} p(x) &= p(x_1, x_2, x_3, \dots, x_T) \\ &= p(x_1)p(x_2 | x_1)p(x_3 | x_1, x_2) \dots \\ &= \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1}) \end{aligned}$$

Probability of the next subpart
given all the previous subparts

Autoregressive Models of Images

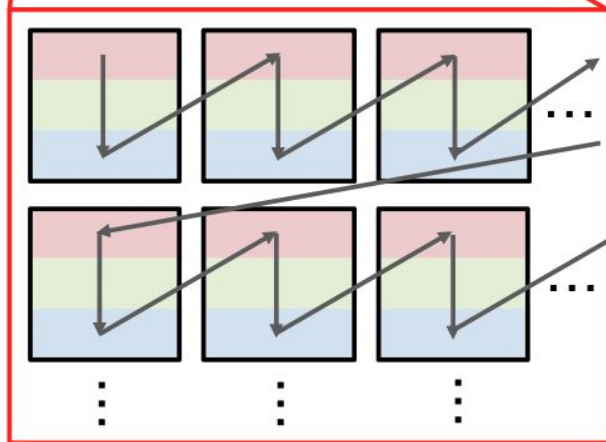
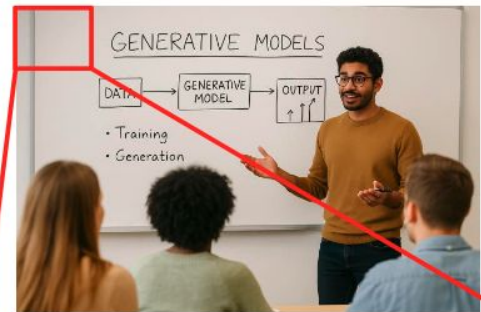
Treat an image as a sequence of 8-bit subpixel values (scanline order)

Predict each subpixel as a classification among 256 values $[0 \dots 255]$

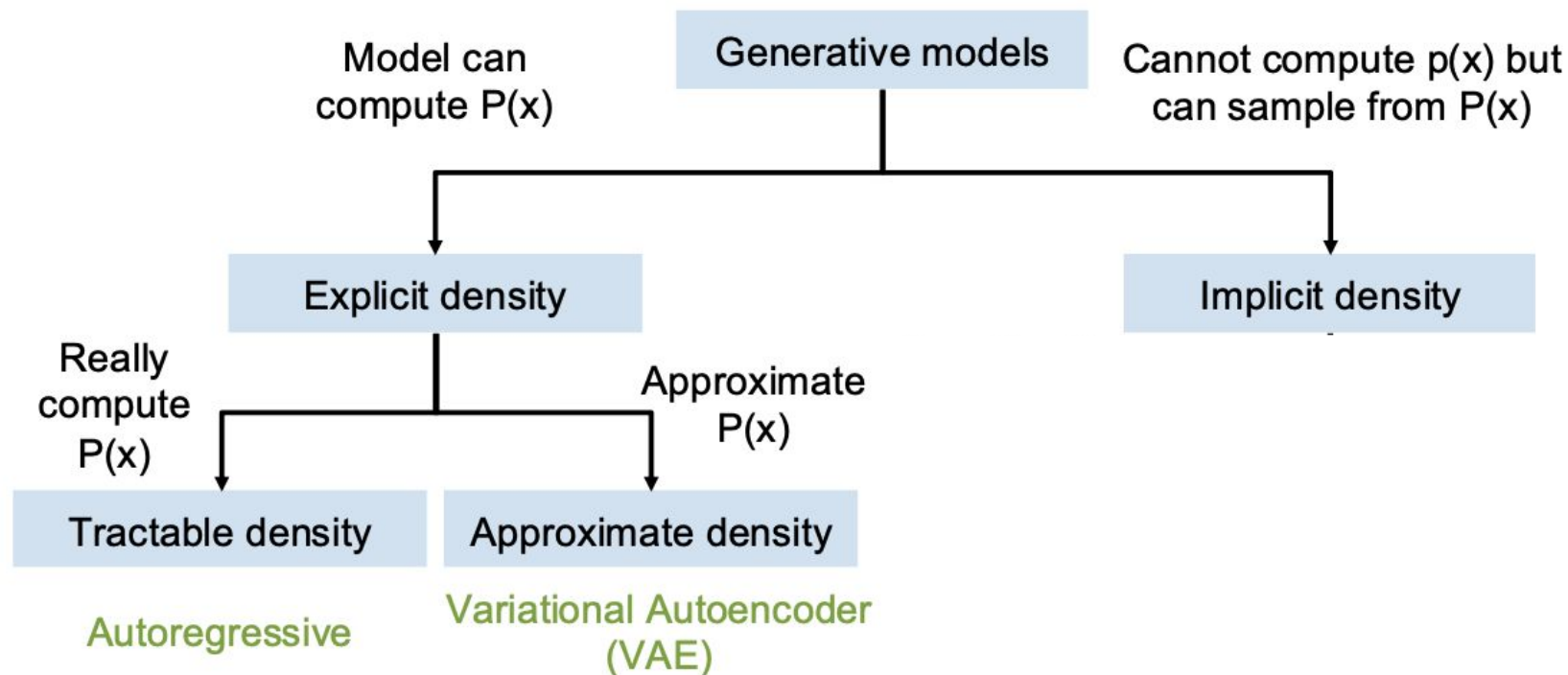
Model with an RNN or Transformer

Problem: Too expensive. 1024×1024 image is a sequence of 3M subpixels

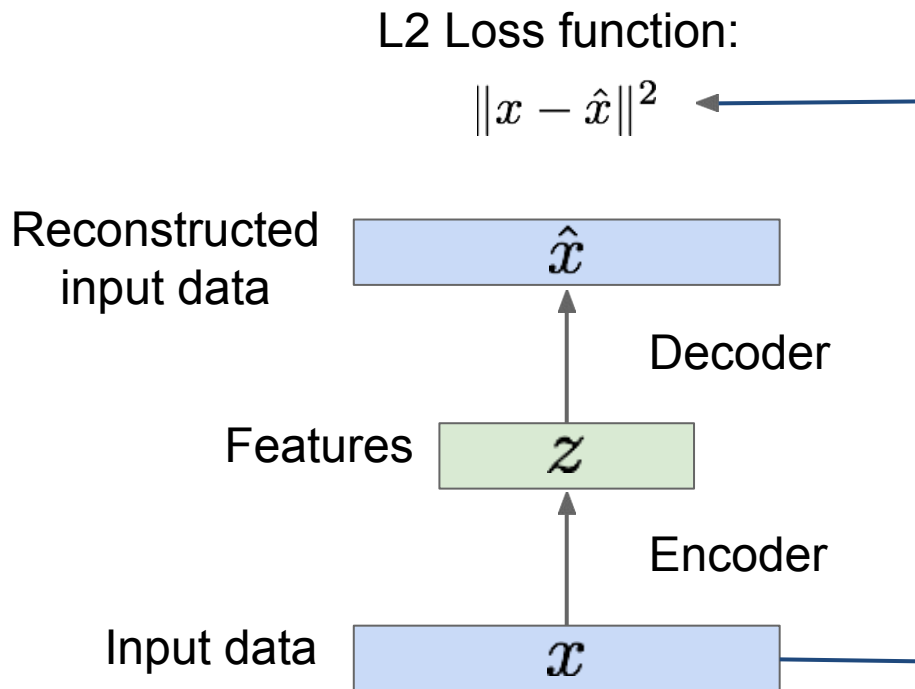
Solution (jumping ahead): Model as sequence of tiles, not sequence of subpixels



Taxonomy of Generative Models



Some background first: AutoEncoders

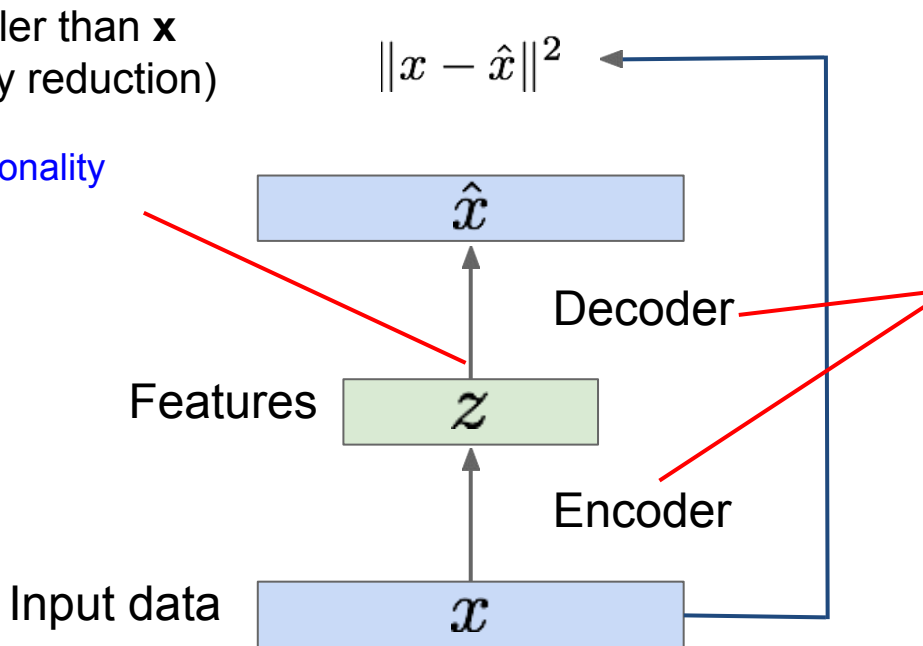


Some background first: AutoEncoders

Unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data

$\mathbf{z}$ usually smaller than $\mathbf{x}$
(dimensionality reduction)

Q: Why dimensionality reduction?



Originally: Linear + nonlinearity (sigmoid)

Later: Deep, fully-connected

Later: ReLU CNN (down / up conv)

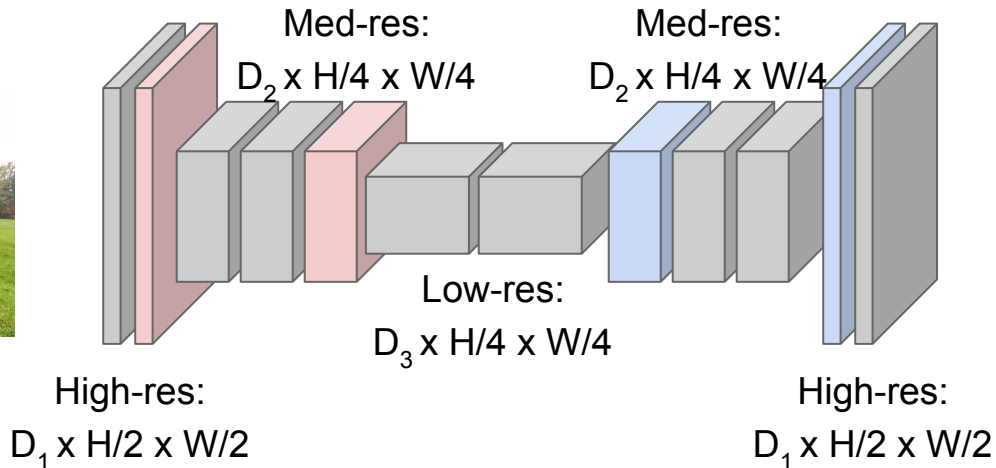
Semantic Segmentation Idea: Fully Convolutional

Downsampling:
Pooling, strided
convolution



Input:
 $3 \times H \times W$

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Upsampling:
???



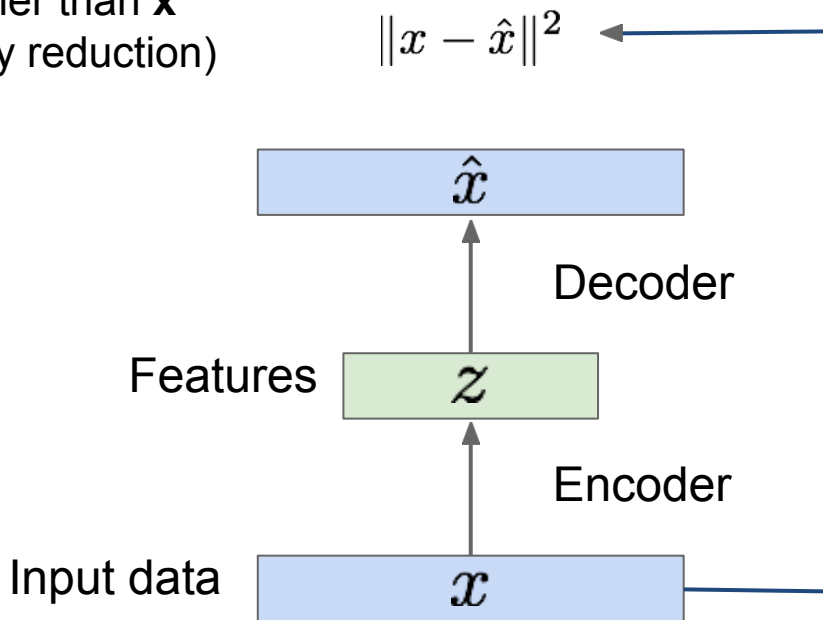
Predictions:
 $H \times W$

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

Some background first: AutoEncoders

Unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data

$\mathbf{z}$ usually smaller than $\mathbf{x}$
(dimensionality reduction)



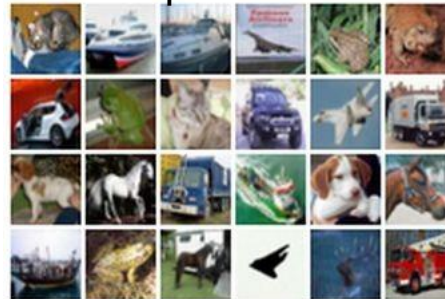
Reconstructed data



Decoder: 4-layer upconv

Encoder: 4-layer conv

Input data

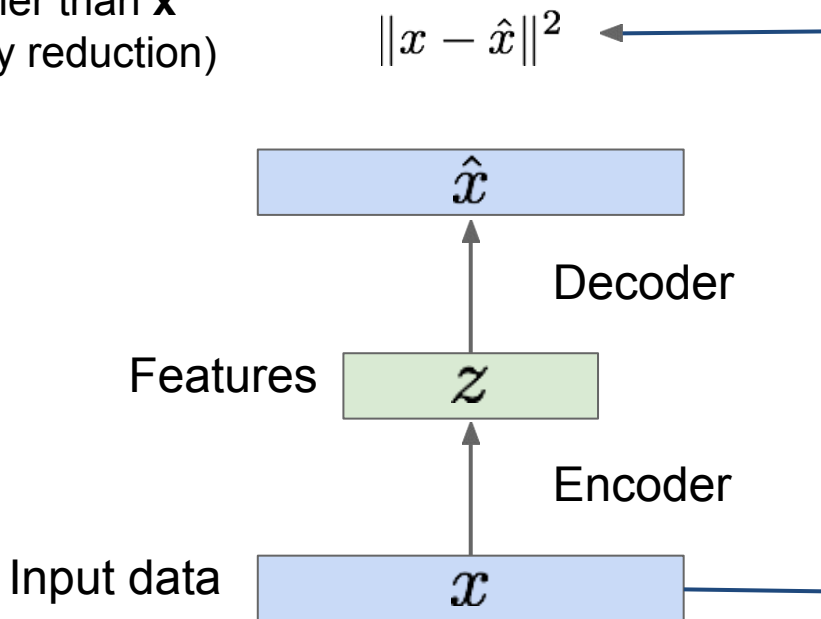


Some background first: AutoEncoders

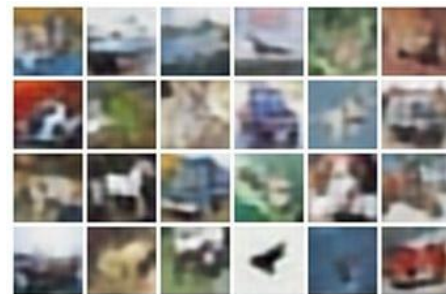
Unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data

Doesn't use labels!

$\mathbf{z}$ usually smaller than $\mathbf{x}$
(dimensionality reduction)



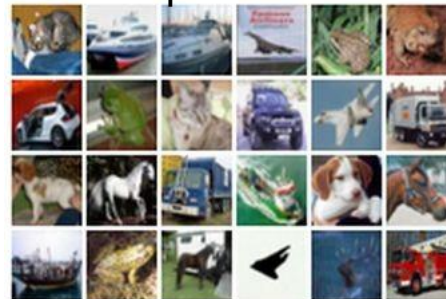
Reconstructed data



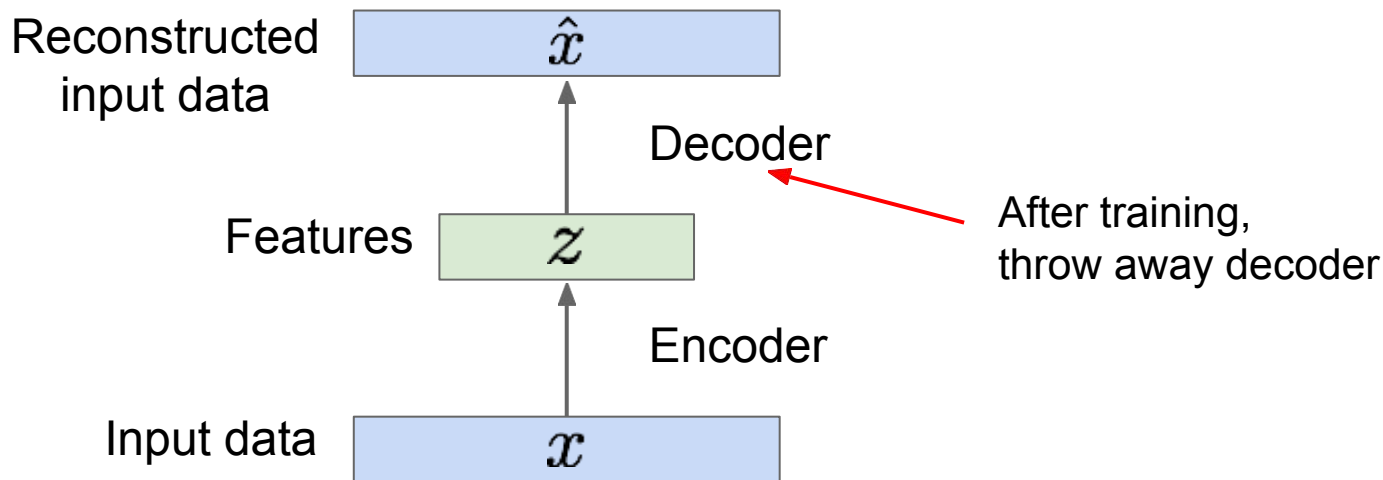
Decoder: 4-layer upconv

Encoder: 4-layer conv

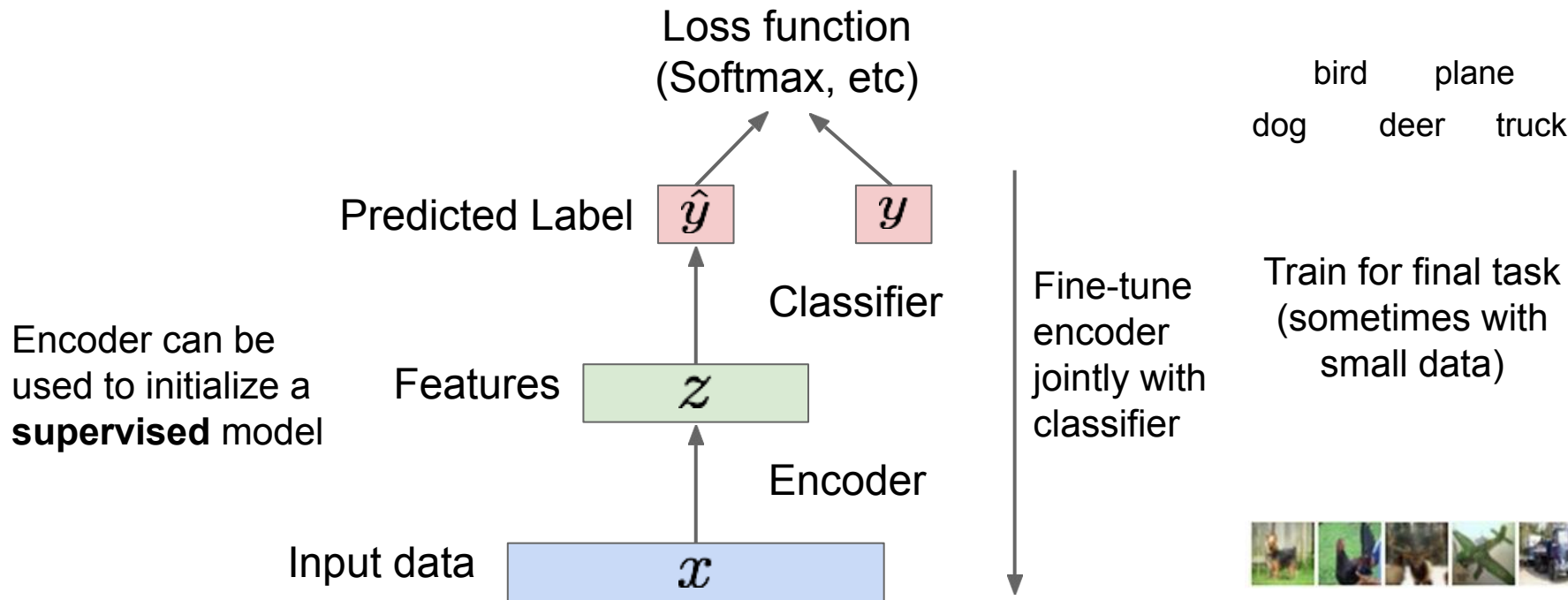
Input data



Some background first: AutoEncoders



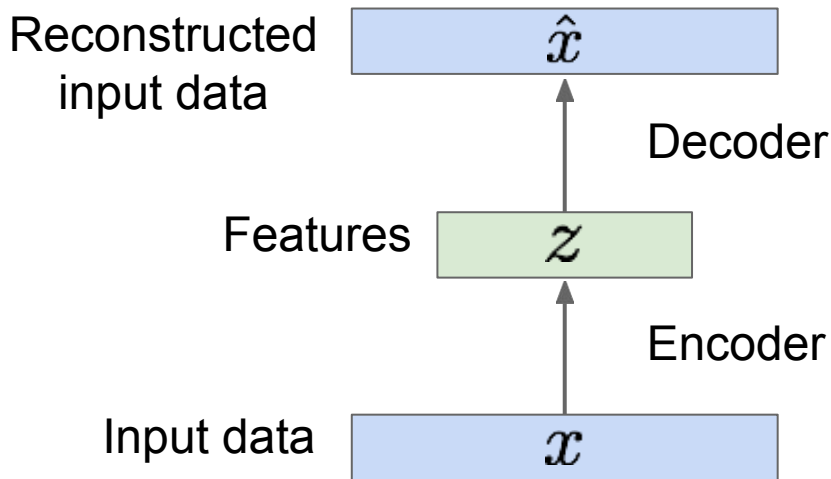
Some background first: AutoEncoders



Some background first: AutoEncoders

Autoencoders can reconstruct data, and can learn features to initialize a supervised model

Features capture factors of variation in training data. Can we generate new images from an autoencoder?



Variational AutoEncoders

Probabilistic spin on autoencoders - will let us sample from the model to generate data!

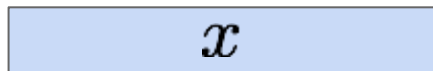
Variational AutoEncoders

Probabilistic spin on autoencoders - will let us sample from the model to generate data!

Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from underlying unobserved (latent) representation $\mathbf{z}$

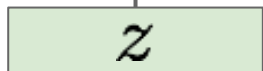
Sample from
true conditional

$$p_{\theta^*}(x \mid z^{(i)})$$



Sample from
true prior

$$p_{\theta^*}(z)$$



Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

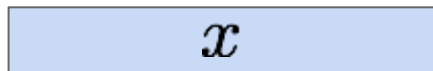
Variational AutoEncoders

Probabilistic spin on autoencoders - will let us sample from the model to generate data!

Assume training data $\{x^{(i)}\}_{i=1}^N$ is generated from underlying unobserved (latent) representation $\mathbf{z}$

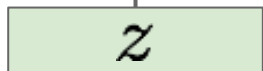
Sample from
true conditional

$$p_{\theta^*}(x \mid z^{(i)})$$



Sample from
true prior

$$p_{\theta^*}(z)$$



Intuition (remember from autoencoders!):
x is an image, **z** is latent factors used to
generate **x**: attributes, orientation, etc.

Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

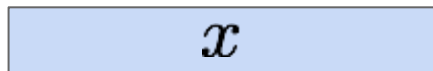
Variational AutoEncoders

We want to estimate the true parameters θ^* of this generative model.

How should we represent this model?

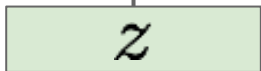
Sample from
true conditional

$$p_{\theta^*}(x \mid z^{(i)})$$



Sample from
true prior

$$p_{\theta^*}(z)$$



Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

Variational AutoEncoders

We want to estimate the true parameters θ^* of this generative model.

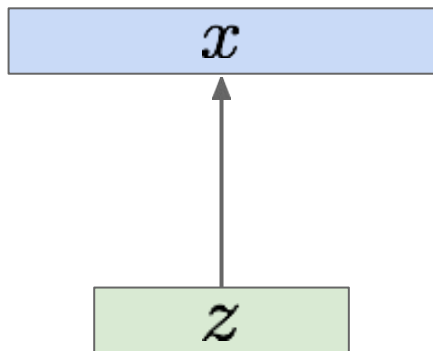
How should we represent this model?

Sample from
true conditional

$$p_{\theta^*}(x | z^{(i)})$$

Sample from
true prior

$$p_{\theta^*}(z)$$



Choose prior $p(z)$ to be simple, e.g. Gaussian. Reasonable for latent attributes, e.g. pose, how much smile.

Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

Variational AutoEncoders

We want to estimate the true parameters θ^* of this generative model.

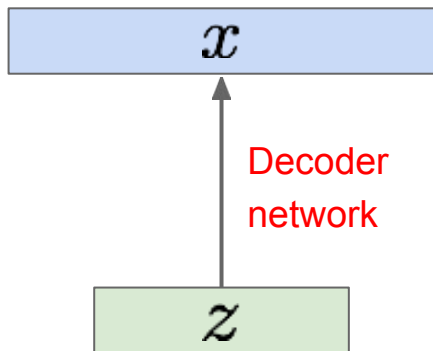
How should we represent this model?

Sample from
true conditional

$$p_{\theta^*}(x | z^{(i)})$$

Sample from
true prior

$$p_{\theta^*}(z)$$



Choose prior $p(z)$ to be simple, e.g. Gaussian. Reasonable for latent attributes, e.g. pose, how much smile.

Conditional $p(x|z)$ is complex (generates image) => represent with neural network

Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

Variational AutoEncoders

We want to estimate the true parameters θ^* of this generative model.

How to train the model?

Learn model parameters to maximize likelihood of training data

$$p_{\theta}(x) = \int p_{\theta}(z)p_{\theta}(x|z)dz$$

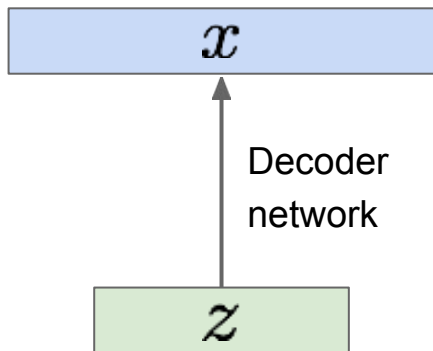
Now with latent z

Sample from
true conditional

$$p_{\theta^*}(x | z^{(i)})$$

Sample from
true prior

$$p_{\theta^*}(z)$$



Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

Variational AutoEncoders

We want to estimate the true parameters θ^* of this generative model.

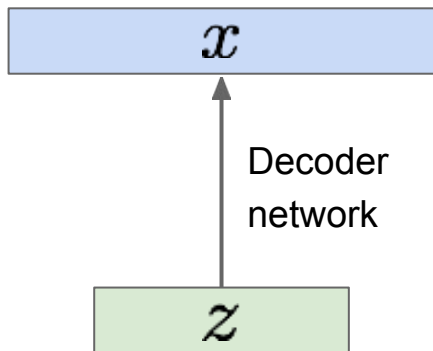
How to train the model?

Sample from
true conditional

$$p_{\theta^*}(x | z^{(i)})$$

Sample from
true prior

$$p_{\theta^*}(z)$$



Learn model parameters to maximize likelihood of training data

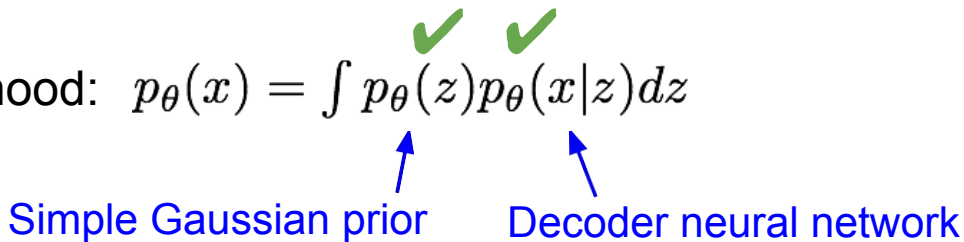
$$p_{\theta}(x) = \int p_{\theta}(z)p_{\theta}(x|z)dz$$

Q: What is the problem with this?

Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

Variational AutoEncoders Intractability

Data likelihood: $p_{\theta}(x) = \int p_{\theta}(z) p_{\theta}(x|z) dz$



Simple Gaussian prior

Decoder neural network

Variational AutoEncoders Intractability

Data likelihood: $p_{\theta}(x) = \int p_{\theta}(z) p_{\theta}(x|z) dz$

↑
Intractable to compute
 $p(x|z)$ for every z !

Solution: In addition to decoder network modeling $p_{\theta}(x|z)$, define additional encoder network $q_{\phi}(z|x)$ that approximates $p_{\theta}(z|x)$

Will see that this allows us to derive a lower bound on the data likelihood that is tractable, which we can optimize

Variational AutoEncoders Intractability

Data likelihood: $p_{\theta}(x) = \int p_{\theta}(z) p_{\theta}(x|z) dz$

↑
Intractable to compute
 $p(x|z)$ for every z !

Solution: In addition to decoder network modeling $p_{\theta}(x|z)$, define additional encoder network $q_{\phi}(z|x)$ that approximates $p_{\theta}(z|x)$

Will see that this allows us to derive a lower bound on the data likelihood that is tractable, which we can optimize

If we can ensure that
 $q_{\phi}(z|x) \approx p_{\theta}(z|x)$,

then we can approximate

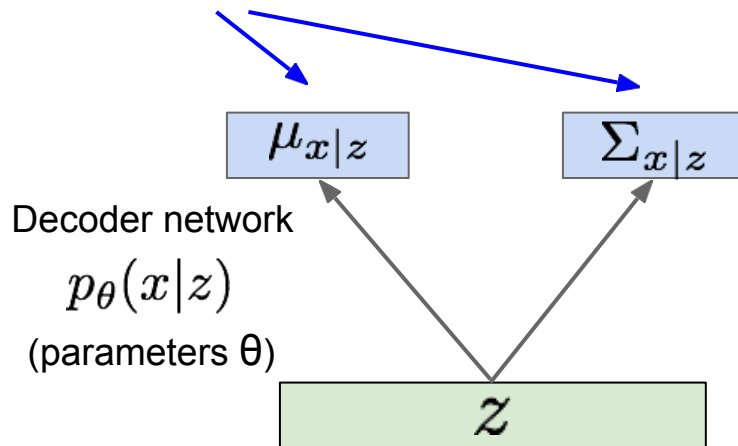
$$p_{\theta}(x) \approx \frac{p_{\theta}(x|z)p(z)}{q_{\phi}(z|x)}$$

Idea: Jointly train both encoder and decoder

Variational AutoEncoders

Since we're modeling probabilistic generation of data, encoder and decoder networks are probabilistic

Mean and (diagonal) covariance of $\mathbf{x} | \mathbf{z}$

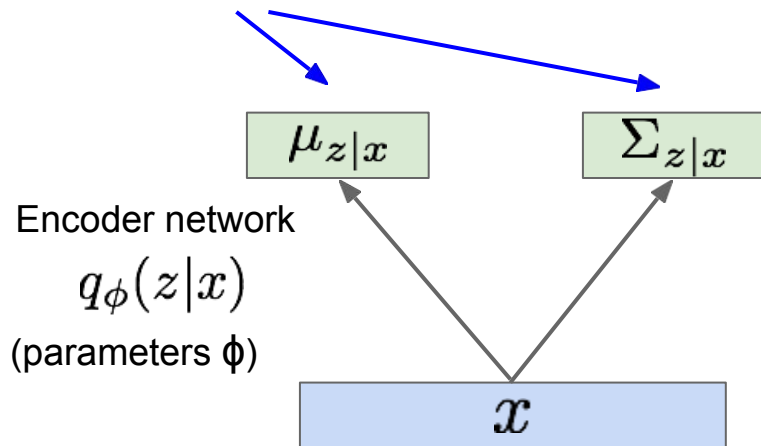


Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

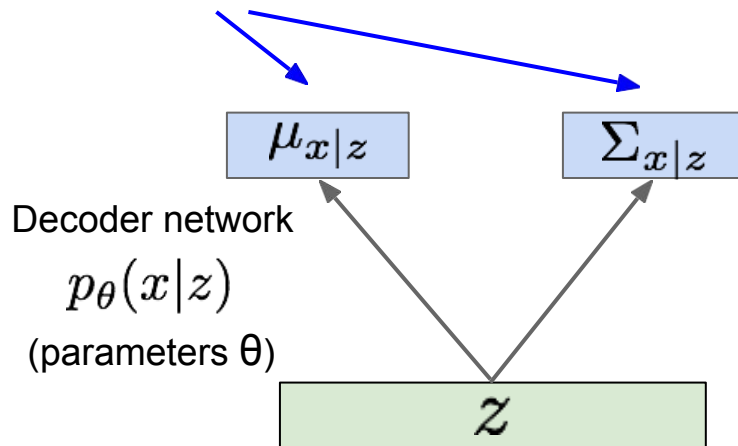
Variational AutoEncoders

Since we're modeling probabilistic generation of data, encoder and decoder networks are probabilistic

Mean and (diagonal) covariance of $\mathbf{z} | \mathbf{x}$



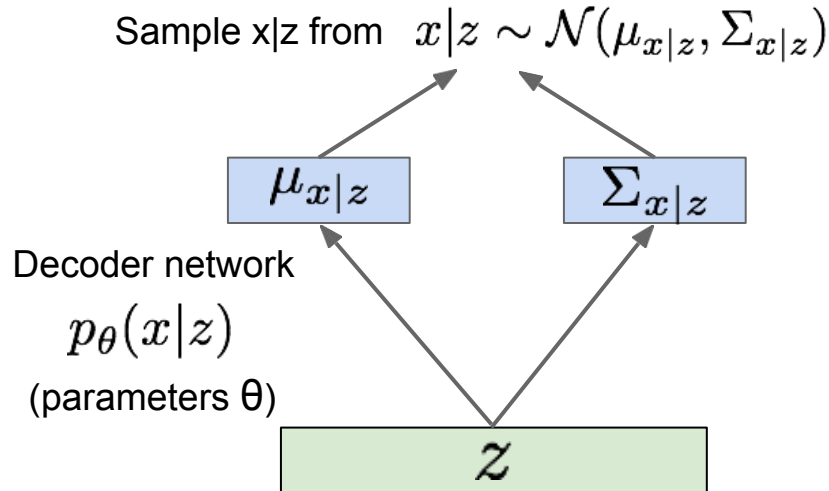
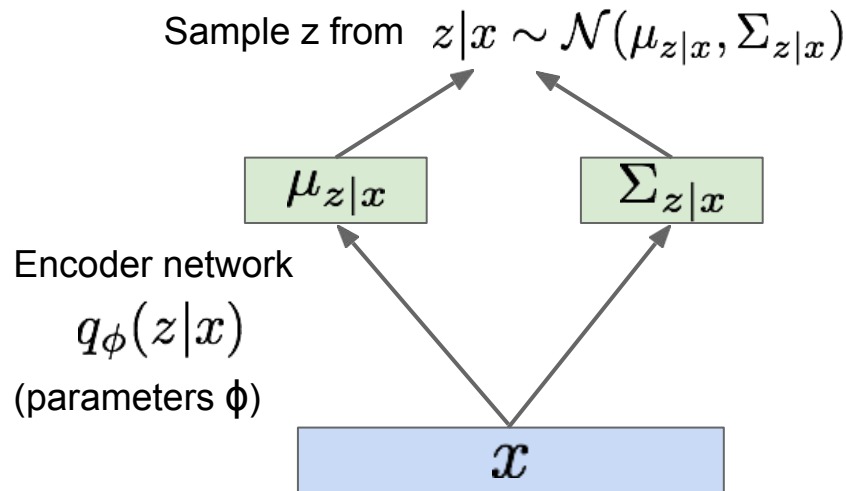
Mean and (diagonal) covariance of $\mathbf{x} | \mathbf{z}$



Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

Variational AutoEncoders

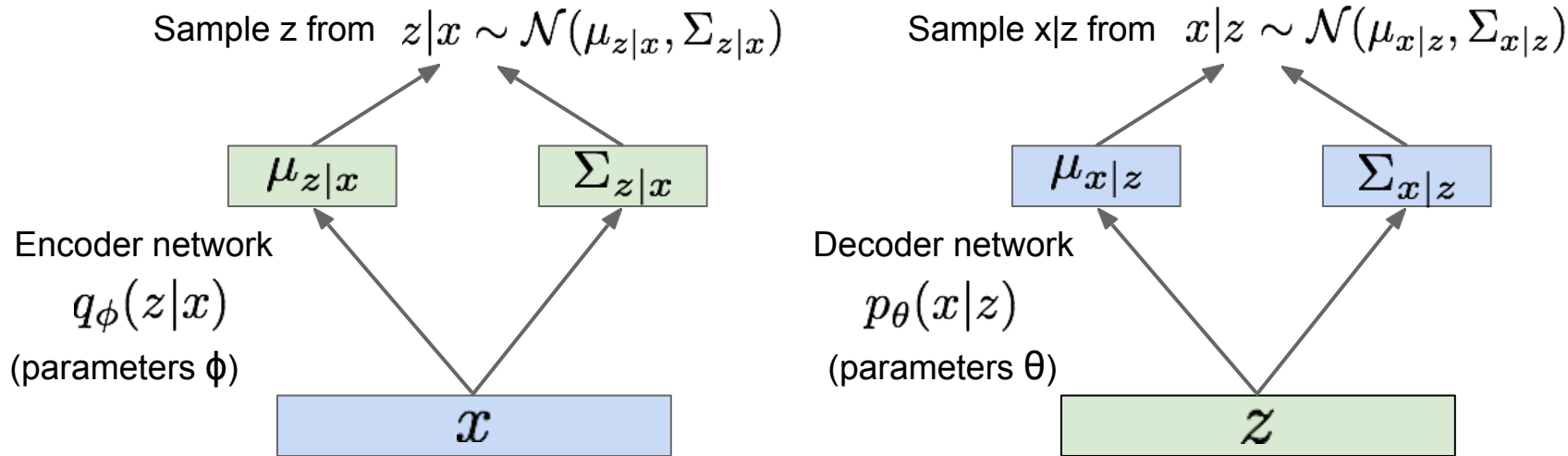
Since we're modeling probabilistic generation of data, encoder and decoder networks are probabilistic



Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

Variational AutoEncoders

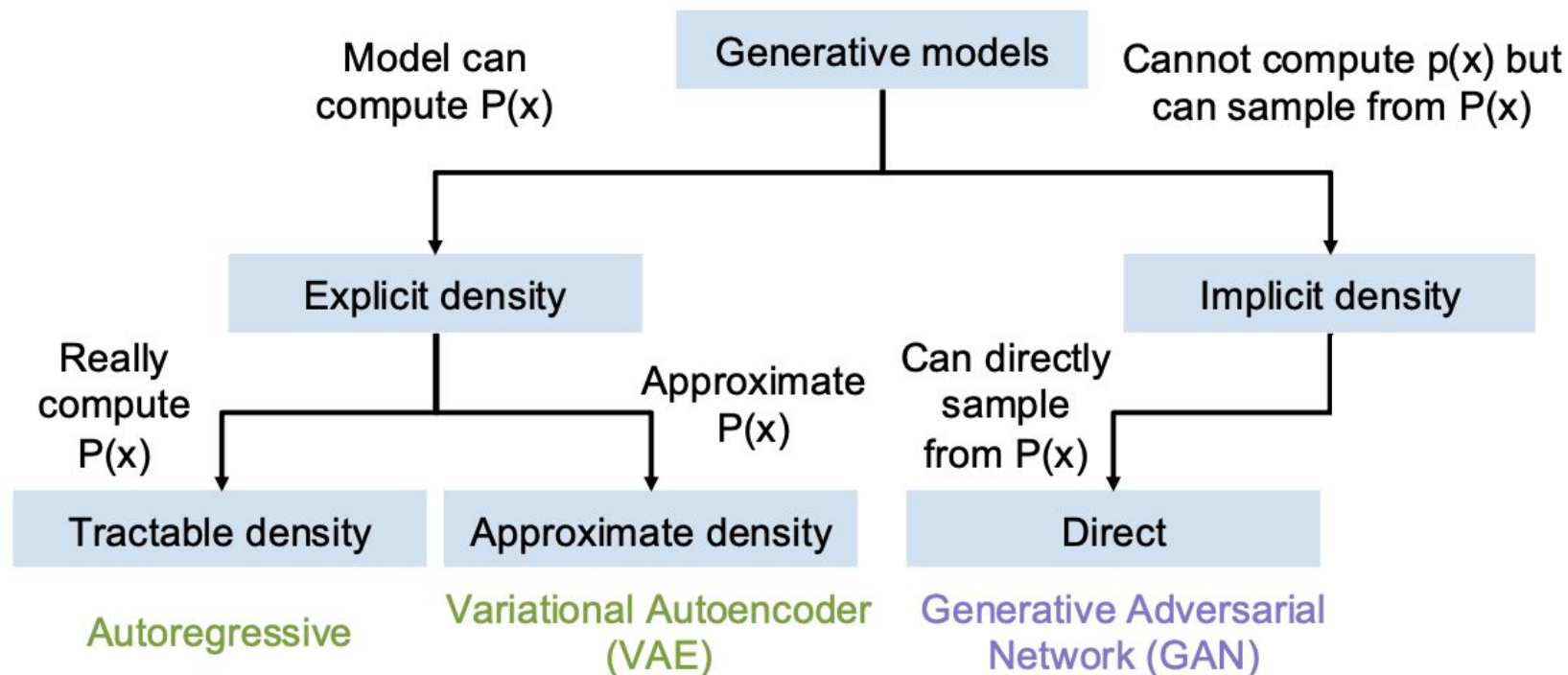
Since we're modeling probabilistic generation of data, encoder and decoder networks are probabilistic



$$p_\theta(x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(z|x)} \approx \frac{p_\theta(x|z)p_\theta(z)}{\boxed{q_\phi(z|x)}}$$

Use **encoder** to compute $q_\phi(z|x) \approx p_\theta(z|x)$

Taxonomy of Generative Models



Generative Adversarial Networks

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Problem: Want to sample from complex, high-dimensional training distribution. No direct way to do this!

Solution: Sample from a simple distribution, e.g. random noise. Learn transformation to training distribution.

Q: What can we use to represent this complex transformation?

Generative Adversarial Networks

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

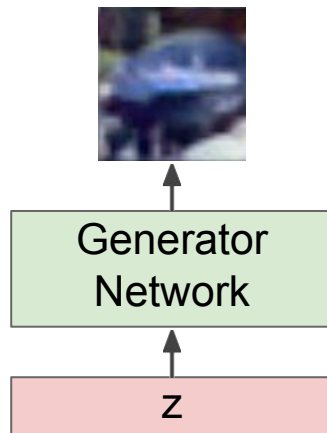
Problem: Want to sample from complex, high-dimensional training distribution. No direct way to do this!

Solution: Sample from a simple distribution, e.g. random noise. Learn transformation to training distribution.

Q: What can we use to represent this complex transformation?

Output: Sample from training distribution

Input: Random noise



Training GANs: two player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Generator network: try to fool the discriminator by generating real-looking images

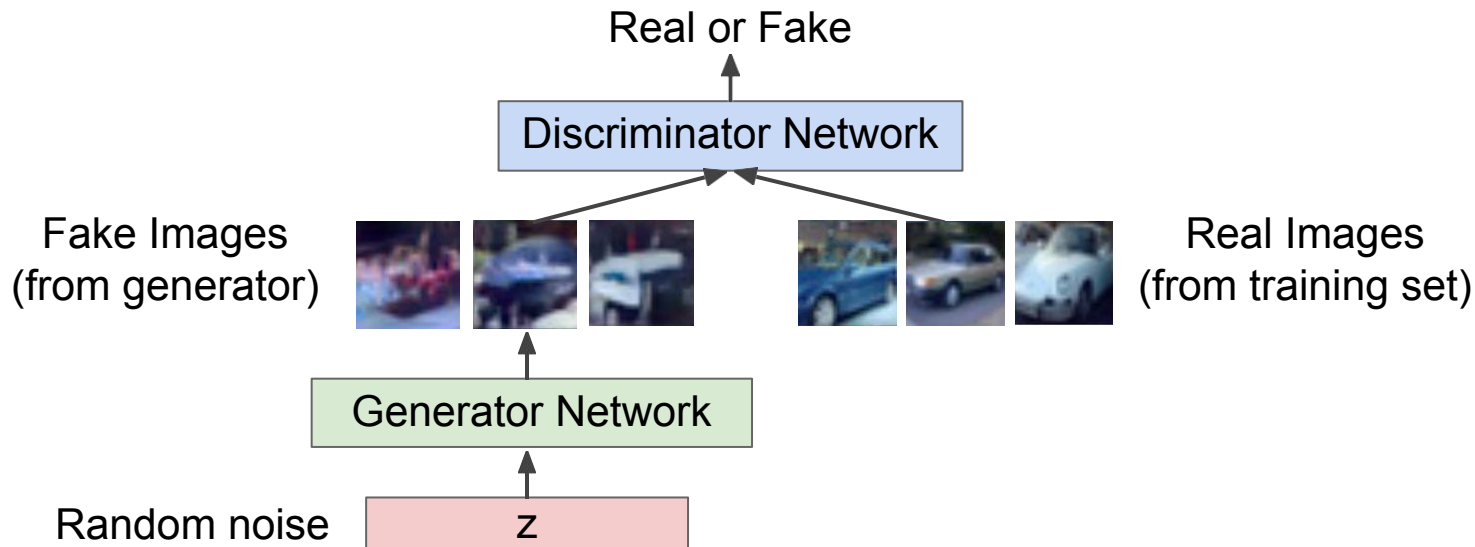
Discriminator network: try to distinguish between real and fake images

Training GANs: two player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Generator network: try to fool the discriminator by generating real-looking images

Discriminator network: try to distinguish between real and fake images



Fake and real images copyright Emily Denton et al. 2015. Reproduced with permission.